

HandVerify: A Comparative Study of Deep Learning Architectures for Writer Verification

Lorenzo Arcioni^{1*}† and Enrico Giordani^{1†}

^{1*}Department of Computer Science, Sapienza University of Rome, Via Salaria 113, Rome, 00198, Italy.

*Corresponding author(s). E-mail(s): arcioni.1885377@studenti.uniroma1.it;

Contributing authors: giordani.2143412@studenti.uniroma1.it;

†These authors contributed equally to this work.

Abstract

Writer verification is a critical biometric task with applications in forensic document analysis, security systems, and authentication. This paper presents HandVerify, a modular framework for offline handwriting verification using deep Siamese neural networks. We systematically evaluate three training objectives—Binary Cross-Entropy (BCE) classification, Contrastive Loss and Triplet Loss—across six ImageNet-pretrained CNN backbones drawn from the ResNet, EfficientNet and MobileNetV3 families, on two benchmark datasets: IAM (English) and RIMES (French). We train and evaluate all 72 (backbone, loss, dataset-pairing) combinations over two within-dataset and two cross-dataset pairings, reporting EER, AUC, decidability and genuine acceptance rate at fixed low false-acceptance rates. The two metric-learning objectives consistently outperform BCE, with Contrastive the most consistent across backbones and splits (paired Wilcoxon signed-rank, $p < 10^{-6}$). The best same-dataset configuration reaches an EER of 3.1% (IAM→IAM) and the best cross-dataset configuration 8.2% (RIMES→IAM), while genuine acceptance degrades sharply at FAR = 0.1%—showing that a low EER does not imply usable operation under strict false-acceptance constraints. Each configuration was trained once, with a single random seed and a single writer-disjoint split; the reported figures are therefore point estimates. All code and experimental protocols are publicly available.

Keywords: Handwriting Verification, Siamese Networks, Contrastive Learning, Triplet Loss, Computer Vision, Deep Learning, Biometric Authentication, Transfer Learning

1 Introduction

Handwriting is a widely studied behavioral biometric, relevant to forensic document examination, signature authentication, and manuscript attribution. As a learned motor skill rather than a fixed physiological trait, it combines strong inter-writer distinctiveness with non-trivial intra-writer variability, making automated *writer verification* – deciding whether two handwriting samples share an author – both valuable and challenging.

Verification differs from identification in that it is formulated as a one-to-one comparison between a probe sample and a claimed identity, rather than as a one-to-many search over a fixed set of enrolled identities. In writer-independent settings, the system must therefore generalize to writers not observed during training, making conventional closed-set identity classification unsuitable as the primary decision mechanism. This motivates pairwise and metric-learning formulations, with Siamese[1] architectures representing a widely adopted paradigm for biometric verification, including face and signature verification.

Despite this maturity, applications to *offline* (image-based) handwriting verification remain fragmented: prior studies typically fix a single backbone or a single training objective, rarely test cross-dataset generalization – arguably the most operationally relevant scenario – and seldom compare loss functions under controlled conditions, even though binary cross-entropy, contrastive, and triplet formulations encode substantially different notions of embedding-space structure.

We address these gaps with **HandVerify**, a modular Siamese framework enabling a systematic comparison along two axes: backbone encoder and training objective. We evaluate six CNN backbones across three families –

ResNet[2]-18/34, EfficientNet[3]-B0/B1, MobileNetV3[4]-Small/Large – all ImageNet[5]-pretrained and adapted to grayscale input via channel-averaging of the first convolutional layer, preserving pretrained knowledge despite the RGB-to-grayscale modality shift. We compare three training paradigms: BCE, which frames verification as direct pairwise classification; Contrastive Loss[6], which pulls genuine pairs together and separates impostor pairs by a margin; and Triplet Loss[7], which enforces relative anchor-positive/anchor-negative separation. We further introduce two tailored training strategies: *epoch-wise dynamic negative resampling*, addressing the combinatorial imbalance between scarce genuine pairs and abundant impostor pairs while exposing the model to diverse negatives; and *selective backbone-layer freezing*, balancing retention of pretrained features against task-specific adaptation on limited data.

The framework is evaluated on IAM[8] and RIMES[9] under four train→test pairings – two within-dataset (IAM→IAM, RIMES→RIMES) and two cross-dataset (IAM→RIMES, RIMES→IAM) – for 72 trained (backbone, loss, pairing) configurations in total. This design lets us assess not only the best-performing combination, but also how consistently a loss function’s advantage holds across backbones and domain shift. Performance is measured via EER, AUC, the decidability index d' , and fixed-operating-point classification metrics, complemented by paired Wilcoxon signed-rank testing for statistical consistency.

The paper is organized as follows. Section 3 presents the architecture, loss formulations, and training strategies. Section 4 describes datasets, preprocessing, and experimental protocol. Section 5 reports the 72-configuration sweep, including same- and cross-dataset performance, significance analysis, and failure-case inspection. Section 6 summarizes findings, practical implications, limitations, and future directions.

2 Related Work

2.1 Traditional Handwriting Verification

Early writer verification relied on handcrafted descriptors extracted from the handwritten trace. Three families dominated: *grapheme-based* methods, which model the statistical distribution of elementary stroke shapes [10]; *texture-based* methods, which treat handwriting as an oriented texture and encode it with Gabor filters or grey-level co-occurrence statistics [11]; and *contour-based* methods, which measure slant, curvature and shape along ink boundaries [12]. Textural and allographic features were combined into strong text-independent baselines by Bulacu and Schomaker [13], within the broader framing of the handwriting-recognition problem surveyed by Plamondon and Srihari [14]. These approaches perform well in controlled settings but degrade under writing variability and noise and require expert feature engineering that transfers poorly across datasets.

2.2 Deep Learning for Writer Analysis

Convolutional Neural Networks (CNNs) replaced hand-designed descriptors with hierarchical features learned directly from pixels. In writer identification, Fiel and Sablatnig [15] used CNN activations as a per-writer feature vector, while Christlein et al. [16] employed penultimate-layer activations as local descriptors aggregated into a global GMM supervector; the same group subsequently removed the need for writer labels altogether, learning descriptors by unsupervised training on surrogate classes [17]. Later work moved to deeper, fragment-level representations, as in FragNet [18]. These systems, however, target *closed-set identification*: they assign a probe to one of a fixed set of enrolled writers and are evaluated with top- k accuracy or mean average precision (mAP).

They do not address the *open-set verification* question—whether two arbitrary samples share an author—which requires an explicit similarity model and a decision threshold rather than a closed classifier, and which is the setting targeted here.

2.3 Siamese Networks and Metric Learning

Verification is naturally cast as metric learning: a Siamese architecture applies two weight-sharing encoders to an input pair and learns an embedding in which same-writer samples are close and different-writer samples are far apart. The paradigm originates with Bromley et al. [19], who introduced twin networks for signature verification, and was later popularised for one-shot recognition by Koch et al. [1]. Two metric objectives shape the embedding geometry explicitly: the Contrastive Loss of Hadsell et al. [6], which pulls genuine pairs together and pushes impostor pairs beyond a margin, and the Triplet Loss of Schroff et al. [7], which enforces a *relative* constraint—an anchor must be closer to a positive than to a negative by a margin—yielding embeddings well suited to open-set biometrics. In handwriting, triplet CNNs were applied to writer retrieval by Keglevic et al. [20]. A third, non-metric option keeps the Siamese encoder but trains a decision head directly on the pair with Binary Cross-Entropy (BCE), as in the one-shot formulation of Koch et al. [1]; this remains the conventional pairwise-classification baseline. The three objectives therefore compete for the same architecture, each structuring the embedding space differently: BCE optimises a decision head without explicitly constraining the space, whereas the two metric losses act on pairwise geometry directly. To the best of our knowledge, no

prior study compares these three objectives under a single fixed training recipe across a shared set of modern backbones on handwriting data, and that is the gap this work addresses.

2.4 Backbone Architectures and Transfer Learning

The encoder choice governs the quality of the learned representation. Residual connections [2] enabled very deep networks; EfficientNet [3] reached comparable accuracy at a fraction of the parameters through compound scaling; and MobileNetV3 [4] prioritised efficiency via depthwise-separable convolutions. Fine-tuning ImageNet-pretrained [5] backbones is now standard, but the domain gap between natural images and near-binary handwriting makes the optimal backbone and transfer strategy for verification non-obvious—motivating the controlled backbone sweep reported here.

2.5 Evaluation of Biometric Verification

As a biometric authentication task, writer verification must be evaluated with detection-oriented metrics rather than plain accuracy. The Equal Error Rate (EER), the operating point at which the False Acceptance Rate equals the False Rejection Rate, provides a single-number, threshold-independent summary, while the ROC curve and its area (AUC) characterise performance across all thresholds; decidability (d') quantifies the separation between the genuine and impostor score distributions [21]. We adopt this protocol throughout and additionally report operation at fixed low-FAR points, the regime relevant to deployed systems and not captured by the EER alone.

3 Methodology

3.1 Problem Formulation

Writer verification is formulated as a binary classification problem operating on pairs of handwriting images. Given two grayscale handwriting samples $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^{H \times W}$, the objective is to determine whether they originate from the same writer. Formally, we seek to learn a function $f : \mathbb{R}^{H \times W} \times \mathbb{R}^{H \times W} \rightarrow \{0, 1\}$ that maps image pairs to binary labels, where 1 indicates a genuine pair (same writer) and 0 indicates an impostor pair (different writers).

Siamese architectures address this problem by learning an embedding function $g_\theta : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^d$ parametrized by neural network weights θ , which projects images into a d -dimensional embedding space where similarity can be assessed. The verification decision is then based on comparing embeddings $\mathbf{e}_1 = g_\theta(\mathbf{x}_1)$ and $\mathbf{e}_2 = g_\theta(\mathbf{x}_2)$ using an appropriate distance or similarity metric.

3.2 Architecture Overview

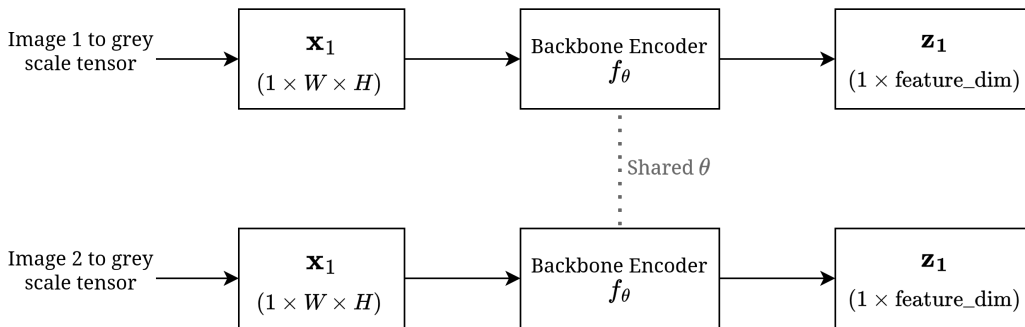


Fig. 1 Overview of the encoder architecture up to the feature dimension.

Our framework implements a modular Siamese architecture composed of three main components: the backbone encoder, the projection head, and the decision mechanism. All components utilize shared weights between the twin networks to ensure symmetric feature extraction.

3.2.1 Backbone Encoder

The backbone encoder extracts high-level discriminative features from input handwriting images. We systematically evaluate six state-of-the-art convolutional neural network architectures organized into three architectural families:

- **ResNet family:** ResNet18 and ResNet34 with residual connections for deep feature learning

- **EfficientNet family:** EfficientNet-B0 and EfficientNet-B1 employing compound scaling strategies
- **MobileNet family:** MobileNetV3-Small and MobileNetV3-Large utilizing depthwise separable convolutions for efficiency

All backbones are initialized with ImageNet pre-trained weights to leverage transfer learning from large-scale visual recognition tasks. To adapt RGB-pretrained models to grayscale handwriting inputs while preserving the learned representations, the first convolutional layer is not reinitialized. Instead, the pretrained convolutional filters corresponding to the three RGB input channels are aggregated into a single-channel representation.

Specifically, given the original first convolutional layer:

$$\text{Conv2d}(3, C_{\text{out}}, \text{kernel_size}, \text{stride}, \text{padding}, \text{bias} = \text{False}), \quad (1)$$

the weights of the three input channels are averaged to produce a single-channel kernel:

$$W_{\text{gray}} = \frac{1}{3} \sum_{c=1}^3 W_c, \quad (2)$$

where W_c denotes the pretrained weights associated with the c -th RGB channel. The resulting weights are then used to define a grayscale-compatible convolutional layer:

$$\text{Conv2d}(1, C_{\text{out}}, \text{kernel_size}, \text{stride}, \text{padding}, \text{bias} = \text{False}), \quad (3)$$

thereby retaining the pretrained knowledge of the original model while enabling processing of single-channel input images.

3.2.2 Head Networks

Depending on the training objective, different head architectures are employed on top of the shared backbone encoder.

Similarity (Decision) Head for BCE Models

For BCE-based Siamese models, verification is formulated as a binary classification problem. Given two input images, the encoder produces two feature embeddings:

$$\mathbf{z}_1, \mathbf{z}_2 \in \mathbb{R}^D \quad (4)$$

where D denotes the embedding dimensionality. The two embeddings are concatenated to form the input representation of the decision head:

$$\mathbf{h}^{(0)} = [\mathbf{z}_1; \mathbf{z}_2] \in \mathbb{R}^{2D} \quad (5)$$

The decision head is implemented as a fully connected network with progressively reduced dimensionality:

$$\mathbb{R}^{2D} \rightarrow \mathbb{R}^{32} \rightarrow \mathbb{R}^{16} \rightarrow \mathbb{R}^8 \rightarrow \mathbb{R}^4 \rightarrow \mathbb{R}^2 \rightarrow [0, 1] \quad (6)$$

Each hidden layer applies a linear transformation followed by batch normalization[22], ReLU activation, and dropout regularization. The generic transformation is defined as:

$$\mathbf{h}^{(l+1)} = \text{Dropout}_{p_l} \left(\text{ReLU} \left(\text{BN} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l)} + \mathbf{b}^{(l)} \right) \right) \right) \quad (7)$$

where:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l+1} \times d_l}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{d_{l+1}} \quad (8)$$

and d_l and d_{l+1} denote the input and output dimensionalities of the l -th fully connected layer.

The dropout probabilities applied after each hidden layer are:

$$p_l \in \{0.20, 0.20, 0.18, 0.16, 0.12\} \quad (9)$$

After the final hidden representation:

$$\mathbf{h}^{(5)} \in \mathbb{R}^2 \quad (10)$$

the last linear layer produces a scalar logit:

$$a = \mathbf{W}^{(5)} \mathbf{h}^{(5)} + b^{(5)} \quad (11)$$

where:

$$\mathbf{W}^{(5)} \in \mathbb{R}^{1 \times 2}, \quad b^{(5)} \in \mathbb{R} \quad (12)$$

The final similarity score is obtained through a sigmoid activation:

$$\hat{y} = \sigma(a) = \frac{1}{1 + e^{-a}} \quad (13)$$

where:

$$\hat{y} \in (0, 1) \quad (14)$$

represents the predicted probability that the two input images belong to the same writer.

Unlike metric-learning approaches, this BCE-based decision head does not explicitly enforce geometric constraints on the embedding space. Instead, it learns a discriminative similarity function directly from the concatenated encoder representations, as illustrated in Figure 2.

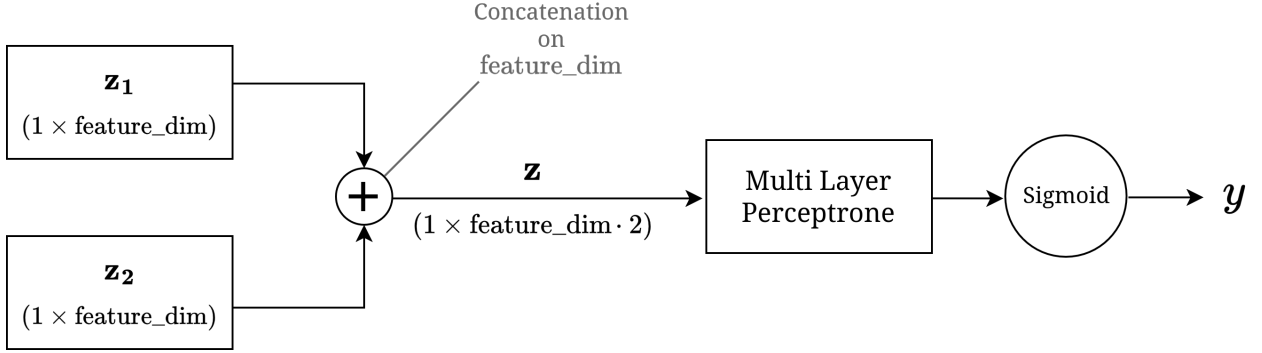


Fig. 2 Similarity (decision) head for BCE-based models. The concatenated embeddings $\mathbf{z}_1$ and $\mathbf{z}_2$ are processed by a multi-layer perceptron followed by a sigmoid activation, producing a similarity score $\hat{y} \in (0, 1)$.

Projection Head for Contrastive and Triplet Models

For contrastive and triplet learning objectives, the shared encoder extracts feature representations from two input images:

$$\mathbf{z}_1, \mathbf{z}_2 \in \mathbb{R}^D \quad (15)$$

where D denotes the dimensionality of the backbone features. The two feature vectors are independently processed by the same projection head with shared parameters:

$$\mathbf{e}_1 = g_\phi(\mathbf{z}_1), \quad \mathbf{e}_2 = g_\phi(\mathbf{z}_2) \quad (16)$$

where g_ϕ denotes the MLP projection head and ϕ represents its learnable parameters, as depicted in Figure 3.

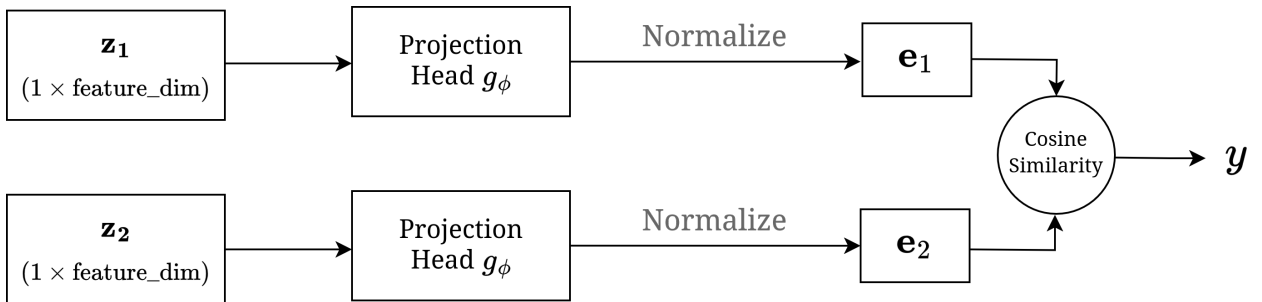


Fig. 3 Projection head for contrastive and triplet learning models. Backbone features $\mathbf{z}_1$ and $\mathbf{z}_2$ are independently mapped by the shared projection head g_ϕ , ℓ_2 -normalized, and compared via cosine similarity.

The projection network maps the backbone features into a lower-dimensional embedding space according to:

$$\mathbb{R}^D \rightarrow \mathbb{R}^{1024} \rightarrow \mathbb{R}^{512} \rightarrow \mathbb{R}^{256} \rightarrow \mathbb{R}^{32} \quad (17)$$

Each hidden layer applies a linear transformation followed by batch normalization, ReLU activation, and dropout regularization:

$$\mathbf{h}^{(l+1)} = \text{Dropout}_{p_l} \left(\text{ReLU} \left(\text{BN} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l)} + \mathbf{b}^{(l)} \right) \right) \right) \quad (18)$$

where:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l+1} \times d_l}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{d_{l+1}} \quad (19)$$

The final projection layer produces the 32-dimensional embedding without applying any non-linear activation:

$$\mathbf{e}_i = \mathbf{W}^{(3)} \mathbf{h}_i^{(3)} + \mathbf{b}^{(3)}, \quad i \in \{1, 2\} \quad (20)$$

The resulting embeddings are normalized independently using ℓ_2 normalization:

$$\hat{\mathbf{e}}_i = \frac{\mathbf{e}_i}{\|\mathbf{e}_i\|_2}, \quad i \in \{1, 2\} \quad (21)$$

obtaining unit-length embeddings:

$$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2 \in \mathbb{R}^{32}, \quad \|\hat{\mathbf{e}}_1\|_2 = \|\hat{\mathbf{e}}_2\|_2 = 1 \quad (22)$$

The dropout probabilities applied after the hidden layers are:

$$p_l \in \{0.20, 0.20, 0.16\} \quad (23)$$

The two normalized embeddings are then compared through cosine similarity:

$$s_{12} = \frac{\hat{\mathbf{e}}_1^T \hat{\mathbf{e}}_2}{\|\hat{\mathbf{e}}_1\|_2 \|\hat{\mathbf{e}}_2\|_2} \quad (24)$$

Since the embeddings are normalized:

$$s_{12} = \hat{\mathbf{e}}_1^T \hat{\mathbf{e}}_2 \quad (25)$$

For contrastive learning, the similarity score is optimized directly by minimizing the distance between genuine pairs and enforcing a margin separation between impostor pairs.

For triplet learning, the same shared embedding function is applied to additional samples during training. The resulting embeddings are compared through relative similarities, enforcing that the anchor-positive similarity is larger than the anchor-negative similarity by a margin:

$$s_{ap} > s_{an} + m \quad (26)$$

Therefore, both approaches use the same shared projection architecture and produce normalized embeddings suitable for metric comparison. The difference lies only in the optimization objective: contrastive learning operates on pairs of embeddings, whereas triplet learning optimizes relative relationships among anchor, positive, and negative samples.

3.3 Loss Functions

We investigate three loss formulations representing different approaches to the verification problem:

3.3.1 Binary Cross-Entropy Loss

BCE treats verification as direct similarity classification:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (27)$$

where $\hat{y}_i \in [0, 1]$ is the predicted similarity score and $y_i \in \{0, 1\}$ is the ground truth label. This formulation provides stable gradients and reliable convergence but does not explicitly structure the embedding space according to metric properties.

3.3.2 Contrastive Loss

Contrastive Loss explicitly optimizes pairwise distances with margin-based separation:

$$\mathcal{L}_{\text{Contrastive}} = \frac{1}{N} \sum_{i=1}^N [y_i d_i^2 + (1 - y_i) \max(m - d_i, 0)^2] \quad (28)$$

where $d_i = 1 - s_i$ represents the cosine-based distance derived from the cosine similarity s_i , and m is the margin hyperparameter. The cosine similarity is computed between L2-normalized feature embeddings, resulting in $s_i \in [-1, 1]$ and $d_i \in [0, 2]$. Genuine pairs ($y_i = 1$) minimize the distance between embeddings, while impostor pairs ($y_i = 0$) are penalized when their distance falls below the margin threshold m , encouraging separation in the embedding space. We set $m = 1.0$ based on preliminary experiments.

3.3.3 Triplet Loss

Triplet Loss enforces relative distance constraints by optimizing the separation between anchor-positive and anchor-negative pairs. Given a triplet composed of an anchor sample, a positive sample from the same writer, and a negative sample from a different writer, the loss is defined as:

$$\mathcal{L}_{\text{Triplet}} = \frac{1}{N} \sum_{i=1}^N \max(d_{ap}^{(i)} - d_{an}^{(i)} + m, 0) \quad (29)$$

where d_{ap} and d_{an} denote the cosine distances between the anchor-positive and anchor-negative embeddings, respectively. The cosine distance is defined as:

$$d(x, y) = 1 - \cos(x, y) \quad (30)$$

where cosine similarity is computed between L2-normalized embeddings, resulting in $d(x, y) \in [0, 2]$. The loss encourages the distance between samples from the same writer to be smaller than the distance between samples from different writers by at least a margin m . When the constraint

$$d_{ap} + m < d_{an} \quad (31)$$

is satisfied, the contribution to the loss becomes zero, meaning that the triplet already satisfies the desired separation. We set $m = 0.5$ based on preliminary experiments, providing a balance between intra-class compactness and inter-class separation.

3.4 Training Strategies

3.4.1 Dynamic Negative Sampling

Class imbalance poses a significant challenge in writer verification [23]. Given W writers with N_w samples each, the number of genuine pairs scales as $O(W \cdot N_w^2)$ while impostor pairs scale as $O(W^2 \cdot N_w^2)$, creating severe imbalance with impostor pairs vastly outnumbering genuine pairs.

We address this through epoch-wise dynamic negative resampling, a random undersampling strategy applied to the majority class [23]. All genuine pairs are preserved (being the minority class), while impostor pairs are randomly resampled each epoch to maintain a target positive ratio. This strategy exposes the model to diverse negative examples across epochs while maintaining balanced training signals:

Algorithm 1 Dynamic Negative Sampling Strategy

- 1: Generate all genuine pairs $\mathcal{P}_{\text{genuine}}$ from same-writer samples
 - 2: Generate complete pool of impostor pairs $\mathcal{P}_{\text{impostor}}$
 - 3: Compute required impostor count: $N_{\text{imp}} = \lfloor N_{\text{gen}} \cdot (1 - r) / r \rfloor$
 - 4: **for** epoch $e = 1$ to E **do**
 - 5: Sample $\mathcal{P}_{\text{impostor}}^{(e)} \subset \mathcal{P}_{\text{impostor}}$ with $|\mathcal{P}_{\text{impostor}}^{(e)}| = N_{\text{imp}}$
 - 6: $\mathcal{D}^{(e)} = \mathcal{P}_{\text{genuine}} \cup \mathcal{P}_{\text{impostor}}^{(e)}$
 - 7: Shuffle $\mathcal{D}^{(e)}$
 - 8: Train on $\mathcal{D}^{(e)}$
 - 9: **end for**
-

where r is the target positive ratio (set to 0.5 for balanced training).

3.4.2 Selective Layer Freezing

To prevent overfitting on limited handwriting data while maintaining transfer learning benefits, we implement selective layer freezing in the backbone encoder. The first L_f convolutional layers remain frozen with gradients disabled, preserving generic low-level features learned from ImageNet, while deeper layers undergo fine-tuning for task-specific adaptation:

$$\theta = \{\theta_{\text{frozen}}, \theta_{\text{trainable}}\}, \quad \text{where} \quad \frac{\partial \mathcal{L}}{\partial \theta_{\text{frozen}}} = 0 \quad (32)$$

We freeze $L_f = 3$ initial layers for all backbone architectures trained, striking a balance between preserving pretrained knowledge and allowing domain adaptation.

3.4.3 Regularization Techniques

To prevent overfitting, we employ a comprehensive regularization strategy:

- **Graduated Dropout:** Starting from a base rate of 0.2, dropout is scaled by a per-layer factor (1.0, 1.0, 0.9, 0.8, 0.6) so that deeper layers retain more expressiveness. This yields rates of $\{0.20, 0.20, 0.18, 0.16, 0.12\}$ for the five-layer BCE decision head and $\{0.20, 0.20, 0.16\}$ for the three-layer contrastive/triplet projection head.
- **Weight Decay:** Decoupled weight decay (as implemented by AdamW) with coefficient $\lambda = 10^{-4}$ applied to all trainable parameters prevents weight magnitude explosion.
- **Batch Normalization:** Applied after each linear transformation to stabilize training dynamics and provide implicit regularization through mini-batch statistics.
- **Data Augmentation:** Training images undergo random transformations including rotation ($\pm 15^\circ$) and random resized crops (90-110% of original size) to simulate natural writing variations while preserving character structure.

3.5 Optimization

We employ the AdamW[24] optimizer with hyperparameters adapted to model capacity:

$$\text{lr} = \begin{cases} 5 \times 10^{-5} & \text{if } |\theta| > 15 \times 10^6 \\ 1 \times 10^{-4} & \text{otherwise} \end{cases} \quad (33)$$

where $|\theta|$ denotes the total number of trainable parameters. Additional optimizer parameters are set to $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and weight decay $\lambda = 10^{-4}$.

Learning rate scheduling employs ReduceLROnPlateau with factor 0.5 and patience 2 epochs, monitoring validation loss for dynamic adaptation. Early stopping with patience 5 epochs prevents overfitting while allowing sufficient convergence time, terminating training when validation loss fails to improve for the patience period.

4 Experimental Setup

4.1 Datasets

We evaluate our framework on two standard handwriting verification datasets. Due to the modular design, the framework supports any dataset organized with writer subdirectories containing handwriting samples. For this study, we use datasets with varying numbers of writers and samples per writer to assess generalization across different data scales.

Images are preprocessed to grayscale and resized to 448×448 pixels, balancing computational efficiency with fine detail preservation. This resolution is sufficient to capture stroke characteristics while remaining manageable for modern GPUs.

4.2 Data Preprocessing

All datasets undergo a rigorous preprocessing pipeline to normalize images while preserving handwriting characteristics. The pipeline combines Otsu’s binarization, morphological operations, and adaptive cropping strategies tailored to each dataset’s structure. Binarization is used exclusively as an auxiliary guide to *compute* crop coordinates and content masks; the pixel values that are ultimately cropped, resized, and saved to disk always come from the original *grayscale* image, never from the binary mask itself.

In brief: each grayscale image is Otsu-thresholded to obtain a binary mask, which is used to (i) detect IAM guide lines, (ii) locate left-margin and top/bottom black regions, and (iii) compute tight-crop bounding boxes via axial projections. These coordinates are then applied to the grayscale image, which is cropped, centered on a square canvas, and resized to 448×448 with area interpolation. IAM full-form images additionally undergo guide-line detection and removal via morphological opening, with the handwritten region extracted according

to the number of detected lines (falling back to a fixed region when detection is unreliable). RIMES text lines are instead height-normalized per writer and vertically stacked into groups of 5 to form 448×448 batches.

The full mathematical formulation of these steps (Otsu thresholding, projection-based tight cropping, IAM guide-line detection rules, RIMES height normalization and batching, visual examples, and associated fallback-/safeguard conditions) is provided in Appendix A.1, together with the exact parameter values used ($\rho = 0.018$ for RIMES, $\rho = 0.02$ for IAM, padding, trim, and batching constants). All these seemingly arbitrary configuration choices are carefully designed with the final dataset in mind, in particular to avoid issues with the models or loss functions such as having too few positive/negative pairs or too few triplets, which could make training more difficult.

4.3 Dataset Statistics Analysis

After preprocessing, we analyze the distribution of images per author across the two resulting datasets, as this directly informs downstream design choices such as batching strategy and writer-level splitting. Table 1 summarizes the key statistics computed on the preprocessed IAM and RIMES datasets.

Table 1 Summary statistics of images per author for the preprocessed IAM and RIMES datasets.

Dataset	Authors	Tot. Images	Mean/Author	Median/Author	Std/Author	Min/Max
IAM	657	1539	2.34	1.0	3.03	1 / 59
RIMES	1500	5046	3.36	3.0	1.27	1 / 10

IAM comprises 657 authors contributing a total of 1539 processed images, with a mean of 2.34 images per author but a median of only 1.0, indicating a strongly right-skewed distribution; this is confirmed by the high standard deviation (3.03) relative to the mean, and by the presence of an extreme outlier author with 59 images, more than an order of magnitude above the median. RIMES, in contrast, comprises 1500 authors and 5046 total processed images (batches), with mean and median again fairly close (3.36 and 3.0 respectively) and a moderate standard deviation (1.27), with a maximum of 10 images per author. The corresponding per-author histograms and boxplot comparison, which visually confirm this asymmetry (IAM dominated by a large spike at 1–2 images per author with a long sparse tail, RIMES broader and more centrally distributed), are reported in Appendix A.2.

This asymmetry has practical implications for the training pipeline: since IAM images already correspond to full, variable-length handwritten samples (one 448×448 image per crop), no further batching is applied, and the heavy-tailed per-author distribution is instead handled through a writer-level (rather than image-level) train/validation/test split, preventing data leakage across sets. For RIMES, the "images" counted here correspond to the stacked 448×448 batches produced by the line-grouping procedure (Appendix A.1); the broader spread around 3 batches per author, compared to IAM, reflects the larger and more variable number of lines each RIMES writer contributed, combined with the fixed batch size $N_{\text{batch}} = 5$.

4.4 Experimental Protocol

4.4.1 Train/Validation/Test Splitting and Dataset Pairings

Each candidate backbone/loss configuration is evaluated on four train→evaluation dataset pairings (IAM→IAM, IAM→RIMES, RIMES→IAM, RIMES→RIMES), for $6(\text{backbones}) \times 3(\text{losses}) \times 4(\text{pairings}) = 72$ trained models in total.

For the two **within-dataset** pairings (IAM→IAM, RIMES→RIMES), the source dataset is writer-disjointly split into three subsets: 70% training, 10% validation (`val_size` = 0.1, used for model selection / early-stopping monitoring), and 20% test (`test_size` = 0.2, held out and used only for the final comprehensive evaluation), with a fixed random seed of 42.

For the two **cross-dataset** pairings (IAM→RIMES, RIMES→IAM), the model is trained on the *entire* source dataset (all of its writers), while the target dataset is itself split writer-disjointly into a 10% validation subset (monitoring/early stopping) and a 90% test subset, the latter used for the final evaluation reported throughout this paper. This setting assesses the ability of a model trained on one handwriting dataset to generalize to a different handwriting domain and data-collection protocol.

Exact writer and pair counts per split are given in Table 2 (Section 5.1); these counts are confirmed identical across all backbone/loss combinations within a given split, since the writer-level split depends only on the fixed random seed and dataset identity, not on backbone or loss.

For all training configurations, negative samples are resampled every epoch, since the number of possible negative pairs is substantially larger than the number of positive pairs. Resampling therefore allows the model

to observe a wider variety of negative examples throughout training, rather than repeatedly using the same subset of negative pairs.

This choice is also motivated by the intended real-world application: in forensic handwriting and signature examination, cases are not submitted to an examiner at random, but precisely because the authenticity of a signature or document is already in question—an instance of what is known in the forensic statistics literature as an *ascertainment* (or *selection*) bias effect on casework composition [25]¹. As a result, the population of samples an examiner actually evaluates is skewed toward disputed, and often forged, material relative to the general population of signatures. Exposing the model to a wide and varied set of negative examples during training is therefore consistent with this real-world prior: a system intended to support forgery detection should be robust across a broad range of negative (forged) cases, rather than a narrow, fixed subset.

No resampling is applied to the validation or test sets, which remain fixed throughout each experiment. Each model is trained from scratch for every architecture/loss/dataset-pairing combination, with early stopping monitored using the corresponding validation set.

4.4.2 Hyperparameters

All 72 configurations of the main sweep share a single uniform hyperparameter configuration: batch size 16, 3 frozen backbone encoder layers, dropout 0.2, and margins 1.0 (Contrastive) / 0.5 (Triplet), matching the values introduced in Section 3. A complete list of representative hyperparameters, including values not otherwise discussed in the main text (maximum epochs, embedding dimension, data-loading workers, etc), is given in Appendix A.3.

5 Results

5.1 Evaluation protocol

We evaluate six backbones (EfficientNet-B0/B1, MobileNetV3-Small/Large, ResNet-18/34) trained under three losses (BCE, Contrastive, Triplet) on four train→test dataset pairings (Section 4.4.1), resulting in 72 trained configurations in total.

For all configurations, pair generation follows the same procedure. Genuine pairs are generated once and kept fixed throughout training, while impostor pairs are drawn from a larger candidate pool and resampled every epoch. The target positive ratio is set to 50%, so that the number of impostor pairs sampled at each epoch equals the number of available genuine pairs. This produces balanced epoch datasets while allowing the model to observe different negative pairs across epochs. The same pair-generation and resampling strategy is used independently of the backbone, loss function, or dataset split.

Table 2 Evaluation protocol and pair counts for each dataset split. Writer counts refer to the train, validation, and test partitions. For each partition, the number of genuine and impostor pairs is equal, corresponding to the 50% positive-ratio setting. Genuine pairs are fixed, whereas impostor pairs are sampled from a larger candidate pool and resampled every epoch. The same pair-generation procedure is used across all backbone/loss configurations, so pair counts are identical across the 18 configurations within each split.

Split	Train w.	Val w.	Test w.	Gen/Imp (train)	Gen/Imp (val)	Gen/Imp (test)
IAM → IAM	459	66	132	3,262 / 3,262	310 / 310	479 / 479
IAM → RIMES	657	150	1,350	4,051 / 4,051	746 / 746	6,429 / 6,429
RIMES → IAM	1,500	65	592	7,175 / 7,175	207 / 207	3,844 / 3,844
RIMES → RIMES	1,049	150	301	5,023 / 5,023	625 / 625	1,527 / 1,527

For the IAM→IAM split, the pair-generation procedure produces 3,262 fixed genuine training pairs and 310 fixed genuine validation pairs. The corresponding impostor candidate pools contain 589,154 and 12,251 possible pairs, respectively. At each epoch, 3,262 impostors are sampled for training and 310 for validation, yielding 6,524 and 620 pairs per epoch, respectively, with an actual positive ratio of 50%. On the test set, 479 genuine and 479 impostor pairs are evaluated, resulting in 958 evaluation pairs. The same generation and resampling procedure is applied to the other three dataset splits; therefore, the pair counts reported in Table 2 are shared by all 18 backbone/loss configurations within each split.

IAM→IAM is the smallest-identity split (132 test writers, 958 evaluation pairs) and therefore likely carries the widest evaluation uncertainty among the same-dataset splits below; RIMES contributes $\sim 2.3\times$ more writers

¹The cited work discusses this effect in the context of firearms examiner casework; we adopt it here as a conceptually analogous argument, as no equivalent study (at the best of our knowledge) specifically quantifying casework composition exists for forensic handwriting/signature examination.

than IAM overall (1,500 vs. 657), a plausible contributing factor to the asymmetry between the two cross-dataset performances (Section 5.3).

Metrics follow the classical FAR/FRR convention. Classification metrics (accuracy/precision/recall/F1) are computed at the EER threshold, not an independently calibrated one.

As only a single seed is trained per configuration, all figures below are point estimates from a single trained model per (backbone, loss, split); no per-model bootstrap confidence intervals were available for this revision (Section 6.2). Mean $\pm$ std figures in Tables 3–4 are across-backbone summary statistics (Table 5), not evaluation-set confidence intervals.

5.2 Same-dataset performance

Table 3 Same-dataset verification performance (mean $\pm$ std across 6 backbones; Table 5).

Loss	Split	AUC	EER (%)	d'
BCE	IAM→IAM	0.956 ± 0.025	9.0 ± 4.0	3.10
Contrastive	IAM→IAM	0.990 ± 0.004	4.9 ± 1.3	3.64
Triplet	IAM→IAM	0.924 ± 0.029	15.3 ± 3.6	1.84
BCE	RIMES→RIMES	0.734 ± 0.180	30.9 ± 15.3	1.03
Contrastive	RIMES→RIMES	0.927 ± 0.007	12.9 ± 1.1	2.12
Triplet	RIMES→RIMES	0.888 ± 0.014	18.3 ± 1.1	1.69

Contrastive is the best-performing loss by a wide margin on both same-dataset splits, and has the smallest across-backbone variance of the three losses. BCE shows by far the largest across-backbone spread on RIMES→RIMES ($\sigma_{\text{AUC}} = 0.180$ vs. 0.007 for Contrastive), directly attributable to the collapsed-model instability documented in Section 5.5. The single best configuration per split (Table 9) is MobileNetV3-Large + Contrastive on IAM→IAM (AUC = 0.995, EER = 3.1%), and ResNet-18 + Contrastive on RIMES→RIMES (AUC = 0.934, EER = 11.5%): Contrastive wins both at the aggregate and the best-single-model level.

5.3 Cross-dataset generalization

Table 4 Cross-dataset verification performance (mean $\pm$ std across 6 backbones; Table 5).

Loss	Split	AUC	EER (%)	d'
BCE	IAM→RIMES	0.621 ± 0.082	40.2 ± 6.9	0.50
Contrastive	IAM→RIMES	0.738 ± 0.032	32.2 ± 2.4	0.77
Triplet	IAM→RIMES	0.699 ± 0.034	35.5 ± 2.5	0.64
BCE	RIMES→IAM	0.715 ± 0.133	32.9 ± 12.8	0.90
Contrastive	RIMES→IAM	0.950 ± 0.014	11.7 ± 2.3	2.13
Triplet	RIMES→IAM	0.869 ± 0.048	20.4 ± 4.9	1.31

Contrastive again dominates both cross-dataset splits. RIMES→IAM (training on the larger, 1,500-writer pool) clearly outperforms IAM→RIMES under every loss – for Contrastive, AUC 0.950 vs. 0.738 – consistent with source writer-pool size (Table 2) as a contributing factor. Best individual configurations (Table 9): EfficientNet-B1 + Contrastive on IAM→RIMES (AUC = 0.772, EER = 29.6%), EfficientNet-B0 + Contrastive on RIMES→IAM (AUC = 0.967, EER = 8.2%).

Figure 4 summarizes the same-vs-cross generalization gap aggregated across all 6 backbones: BCE shows the largest AUC drop, Contrastive an intermediate one, and Triplet the smallest but also the lowest absolute same-dataset AUC among the three losses.

5.4 Fixed operating points (FAR/FRR trade-off)

Fixed operating points provide a complementary view of verification performance by quantifying the proportion of genuine pairs correctly accepted (GAR) under stringent false-acceptance constraints. In particular, FAR values

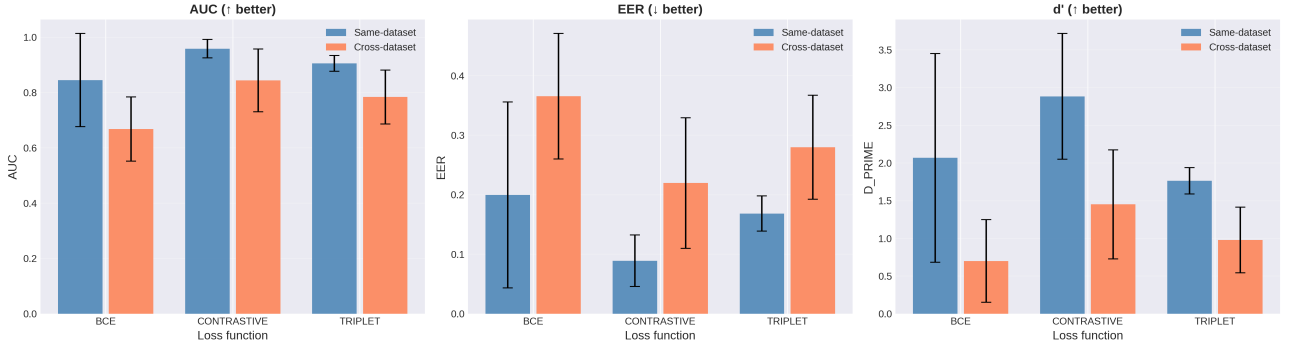


Fig. 4 AUC, EER and d' , same-dataset vs. cross-dataset, aggregated per loss (mean $\pm$ std across 6 backbones).

of 1% and 0.1% represent increasingly conservative operating conditions, making the corresponding GAR values informative of how well a model maintains genuine-user acceptance while limiting false matches. The results also highlight the substantial impact of the dataset shift: while the best same-dataset configuration maintains relatively high GAR at both operating points, cross-dataset performance deteriorates markedly, especially at FAR= 0.1%. This indicates that a high overall discrimination capability does not necessarily translate into robust performance in low-FAR operating regimes, which are particularly relevant for practical biometric verification systems.

Table 5 GAR at fixed FAR, best (backbone, loss) per split (Table 9).

Split	GAR@FAR1%	GAR@FAR0.1%
IAM→IAM (MobileNetV3-Large, Contr.)	87.5%	72.4%
IAM→RIMES (EfficientNet-B1, Contr.)	15.9%	4.6%
RIMES→IAM (EfficientNet-B0, Contr.)	62.6%	13.0%
RIMES→RIMES (ResNet-18, Contr.)	53.9%	11.5%

EER can hide substantial performance differences at strict operating points. For example, the best same-dataset configuration achieves a GAR of 87.5% at FAR = 1% on IAM→IAM, corresponding to a genuine rejection rate of 12.5%. On RIMES→RIMES, the GAR decreases from 53.9% at FAR = 1% to only 11.5% at FAR = 0.1%, corresponding to genuine rejection rates of 46.1% and 88.5%, respectively. Cross-dataset performance is markedly worse, with GAR falling to the single-digit range at FAR = 0.1% for IAM→RIMES (4.6%), corresponding to a genuine rejection rate of 95.4%. These results illustrate that good overall discrimination, as reflected by EER, does not necessarily translate into effective operation under stringent false-acceptance constraints.

5.5 Per-backbone analysis

Collapsed models. Four (backbone, loss, split) configurations are collapsed ($AUC \approx$ chance), all under BCE:

- MobileNetV3-Large / IAM→RIMES: $AUC = 0.528$
- MobileNetV3-Small / IAM→RIMES: $AUC = 0.516$
- MobileNetV3-Small / RIMES→IAM: $AUC = 0.540$
- EfficientNet-B1 / RIMES→RIMES: $AUC = 0.468$

This explains BCE’s large across-backbone variance in Sections 5.2–5.3.

Contrastive-over-BCE AUC gain, averaged across all four splits per backbone: MobileNetV3-Small (+0.241), EfficientNet-B1 (+0.190), ResNet-34 (+0.163), EfficientNet-B0 (+0.122), MobileNetV3-Large (+0.093), ResNet-18 (+0.061). ResNet-18 is the only backbone with zero collapsed BCE configurations and benefits least from switching losses.

Figure 5: across-backbone AUC std is tight under Contrastive throughout (± 0.004 – ± 0.032) but large under BCE on the two splits with collapsed models (RIMES→RIMES ± 0.180 , RIMES→IAM ± 0.133). IAM→RIMES is the hardest cell under every loss ($AUC \leq 0.738$); RIMES→IAM under Contrastive ($AUC 0.950$) is close to same-dataset RIMES→RIMES (0.927) despite being cross-dataset.

Aggregate profile across all losses and splits.

ResNet-18 ranks first on AUC, EER and d' , but has the second lowest GAR@FAR1% (23.3%) and lowest GAR@FAR0.1% (9.8%, near-tied with ResNet-34) of all six backbones. Overall separability and low-FAR-tail

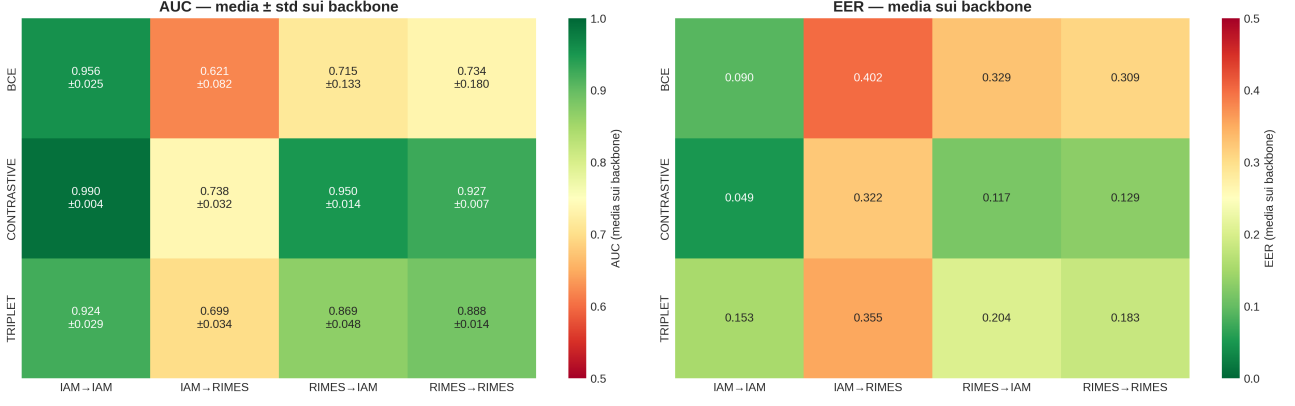


Fig. 5 AUC and EER, mean across the 6 backbones, per loss $\times$ split combination.

Table 6 Per-backbone verification metrics, averaged across all 12 loss $\times$ split combinations.

Backbone	AUC	EER (%)	d'	GAR@FAR1%	GAR@FAR0.1%
ResNet-18	0.869	19.2	1.84	23.3%	9.8%
EfficientNet-B0	0.849	20.2	1.75	25.3%	10.0%
EfficientNet-B1	0.838	21.1	1.77	31.0%	14.3%
MobileNetV3-Large	0.833	22.3	1.59	27.1%	13.1%
ResNet-34	0.828	23.1	1.52	20.2%	9.8%
MobileNetV3-Small	0.788	26.3	1.38	25.0%	14.8%

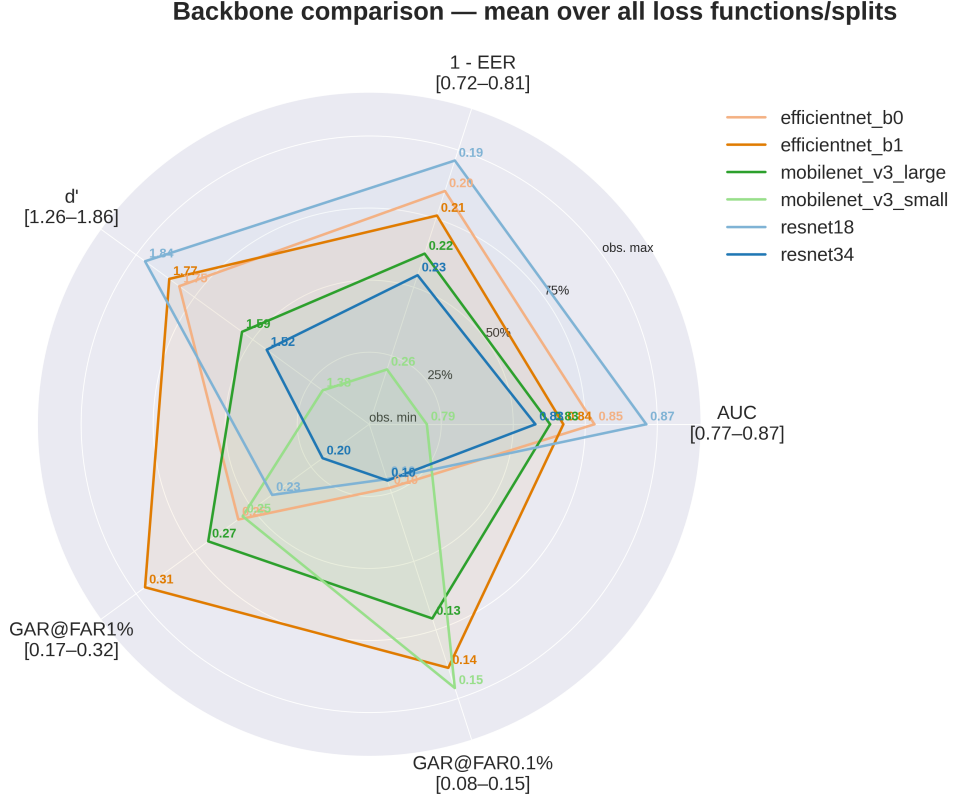


Fig. 6 Per-backbone verification profile, averaged across all loss functions and dataset splits. Axis ranges scaled between observed min/max per metric; annotated values are raw means (Table 6). Color by architecture family (blue: ResNet, orange: EfficientNet, green: MobileNetV3). Darker shade = larger variant.

behavior are therefore decoupled for this backbone: a small number of high-scoring impostor pairs depresses GAR at strict thresholds without affecting AUC/EER, consistent with the boundary-saturated impostor tail reported for ResNet-18 under Contrastive on cross-dataset splits (Section 5.10).

EfficientNet-B1 shows the inverse profile: third on AUC, second on d' and third on EER, but highest GAR@FAR1% (31.0%, +7.7 points over ResNet-18).

MobileNetV3-Small has the worst AUC, EER and d' of all six backbones, consistent with its two collapsed BCE configurations, yet the highest GAR@FAR0.1% (14.8%) — an isolated-threshold effect, not a discriminability advantage.

Within the ResNet family, ResNet-34 underperforms ResNet-18 on every metric except GAR@FAR0.1% (near-tied, 9.8% vs. 9.8%), despite twice the parameter count; no comparable inversion occurs between EfficientNet-B0/B1. ResNet-34 is the only backbone that leads on no metric in Table 6.

Taken together, these results lead us to select ResNet-18 as the backbone for the subsequent analyses.

5.6 Paired analysis of loss-function performance

Table 7 Paired Wilcoxon signed-rank test on AUC ($n = 24$: 6 backbones $\times$ 4 splits).

Comparison	Mean AUC (A)	Mean AUC (B)	W	p	Effect size
BCE vs. Contrastive	0.756	0.901	1.0	2.38×10^{-7}	-0.993
BCE vs. Triplet	0.756	0.845	63.0	0.0115	-0.580
Contrastive vs. Triplet	0.901	0.845	0.0	1.19×10^{-7}	+1.000

Because the 24 observations are not fully independent experimental replicates—they share the same backbone architectures, dataset splits, and underlying data, and each configuration is represented by a single training run—the Wilcoxon tests are interpreted as paired analyses of the consistency of loss-function differences across configurations, rather than as evidence from 24 independent training experiments. Under this interpretation, all three pairwise comparisons are statistically significant at $\alpha = 0.05$. Contrastive significantly outperforms both BCE and Triplet, with very large effect sizes ($\rho = -0.993$ and $\rho = +1.000$, respectively; both $p < 10^{-6}$). Triplet also outperforms BCE, although the effect is substantially smaller ($\rho = -0.580$, $p = 0.0115$). Since the training recipes are uniform across losses (Section 4.4.1), these differences are consistent with an effect of the loss objective itself. However, as discussed in Section 5.5, Contrastive’s overall advantage does not necessarily reflect a uniform improvement across all architectures; part of its gain arises from its greater robustness on architectures for which BCE exhibits higher variability.

5.7 High-precision validation with an exhaustive impostor pool

For ResNet-18 + Contrastive trained on RIMES and evaluated on the *full* IAM impostor pool (4,051 genuine pairs against 1,179,440 impostor pairs, $\sim 291\times$ the balanced protocol’s impostor count), the exhaustive-pool re-evaluation gives:

Table 8 Exhaustive-impostor-pool validation, RIMES→IAM, ResNet-18 + Contrastive, vs. the balanced protocol.

Protocol	Impostor pairs	AUC	EER	FAR@0.1% [†]	GAR@0.1%	FAR@1% [†]	GAR@1%
Balanced	4,051	0.943	13.4%	0.078%	8.0%	0.989%	34.0%
Exhaustive	1,179,440	0.945	13.0%	0.100%	9.6%	1.000%	34.5%

[†] Effective FAR achieved at the threshold selected for the corresponding nominal FAR target (0.1% or 1%), not the nominal target itself.

The exhaustive pool yields results that are essentially in line with the balanced protocol across all metrics (AUC 0.943 $\rightarrow$ 0.945; EER 13.4% $\rightarrow$ 13.0%; GAR@1% 34.0% $\rightarrow$ 34.5%), indicating that 1:1 balanced impostor subsampling in the main sweep was already a reliable estimate for this configuration. At the 1% FAR target, the effective FAR is nearly identical between the two protocols (0.989% for balanced vs. 1.000% for exhaustive), and GAR@1% tracks closely as a result (34.0% $\rightarrow$ 34.5%). At the stricter 0.1% FAR target, the effective FAR achieved is somewhat lower for the balanced protocol (0.078%) than for the exhaustive one (0.100%): with only 4,051 impostor pairs, the balanced protocol cannot finely resolve thresholds in the low-FAR regime, so the selected threshold ends up more conservative than intended, which in turn explains most of the gap in GAR@0.1% (8.0% $\rightarrow$ 9.6%) — the exhaustive pool, precisely by including far more impostor pairs, is able to

hit the 0.1% target much more accurately. Overall, no meaningful degradation in discriminability is observed when the impostor pool is expanded to its full exhaustive size; the residual differences at strict FAR targets are attributable to threshold resolution rather than to overfitting of the balanced subsampling. This is run for a single (backbone, loss, split) triple only, given the cost of exhaustive scoring, and does not revisit the per-backbone (Section 5.5) or Wilcoxon (Section 5.6) analyses.

5.8 Best-model summary

Table 9 Best-performing (backbone, loss) configuration per split.

Split	Best (backbone, loss)	AUC	EER
IAM→IAM	MobileNetV3-Large, Contrastive	0.995	3.1%
IAM→RIMES	EfficientNet-B1, Contrastive	0.772	29.6%
RIMES→IAM	EfficientNet-B0, Contrastive	0.967	8.2%
RIMES→RIMES	ResNet-18, Contrastive	0.934	11.5%

Table 10 Classification metrics at the EER threshold, best (backbone, loss) per split.

Split	Accuracy	Precision	Recall	F1	μ_g/μ_i	σ_g/σ_i
IAM→IAM	0.969	0.969	0.969	0.969	0.893/0.005	0.118/0.277
IAM→RIMES	0.704	0.704	0.704	0.704	0.829/0.648	0.170/0.219
RIMES→IAM	0.918	0.918	0.918	0.918	0.938/0.413	0.110/0.294
RIMES→RIMES	0.885	0.885	0.885	0.885	0.791/0.063	0.284/0.357

As noted in Section 5.1, these figures are computed at the EER threshold of each model on its own evaluation set, not an independently calibrated threshold. The provided ROC/DET/distribution figures (Figures 7–9) cover ResNet-18 across all 12 loss×split combinations, and are shown as a representative supplementary panel rather than as the aggregate-best-per-split configurations of Table 9 (which include MobileNetV3-Large and EfficientNet-B0/B1 for three of the four splits).

The ROC curves in Figure 7 report bootstrap 95% confidence intervals alongside each AUC estimate. Contrastive achieves the highest AUC on all four splits for ResNet-18 (IAM→IAM: 0.990; IAM→RIMES: 0.764; RIMES→IAM: 0.943; RIMES→RIMES: 0.934), consistently outperforming BCE and Triplet across the full FAR range rather than only near the EER operating point. The ordering of BCE and Triplet is not fixed across splits: BCE outperforms Triplet on two of the four splits (IAM→IAM, RIMES→IAM), but Triplet marginally exceeds BCE on IAM→RIMES, the hardest cross-dataset pairing for this backbone, and RIMES→RIMES. Confidence-interval width is governed primarily by evaluation-set size rather than by the same-vs-cross-dataset distinction: the narrowest interval overall belongs to Contrastive/IAM→IAM (width 0.009, $n = 958$ pairs), while BCE/IAM→IAM and BCE/RIMES→RIMES – despite being a same-dataset split – shows the widest intervals (width 0.024 and 0.028, respectively), reflecting their comparatively small test sets relative to the cross-dataset splits (e.g. IAM→RIMES, $n = 12,858$ pairs, CI width 0.016–0.017 across losses). At the shape level, the same-dataset curves (IAM→IAM, RIMES→RIMES) rise sharply toward the top-left corner across all three losses, whereas the cross-dataset curves – particularly IAM→RIMES – remain comparatively close to the chance diagonal over a wide FAR range, visually corroborating the same-vs-cross-dataset generalization gap discussed in Section 5.3.

The DET curves in Figure 8 provide a visual representation of the strong variation in verification performance across losses and dataset splits. The IAM→IAM curves are consistently the most favorable, reaching relatively low FRR at small FAR values across all three losses. However, the shape differs by loss: BCE exhibits a pronounced, nearly flat plateau in the low-FAR region before dropping sharply, Contrastive shows a comparable but noisier and less perfectly flat shelf at a higher FRR level, while Triplet decreases more smoothly and monotonically without a distinct plateau. In contrast, the IAM→RIMES curves remain close to the upper part of the plot over a large portion of the FAR range for all three losses, indicating a substantial loss of discrimination under the cross-dataset shift, with a sharp drop only near each curve’s EER point. RIMES→IAM shows intermediate behavior between the two same-dataset splits and IAM→RIMES, with Contrastive (EER = 13.4%) clearly outperforming BCE (EER = 18.2%) and Triplet (EER = 22.6%). The RIMES→RIMES curves, evaluated on the second same-dataset split, also show substantial variation across losses, with Contrastive

Curve ROC — backbone: resnet18

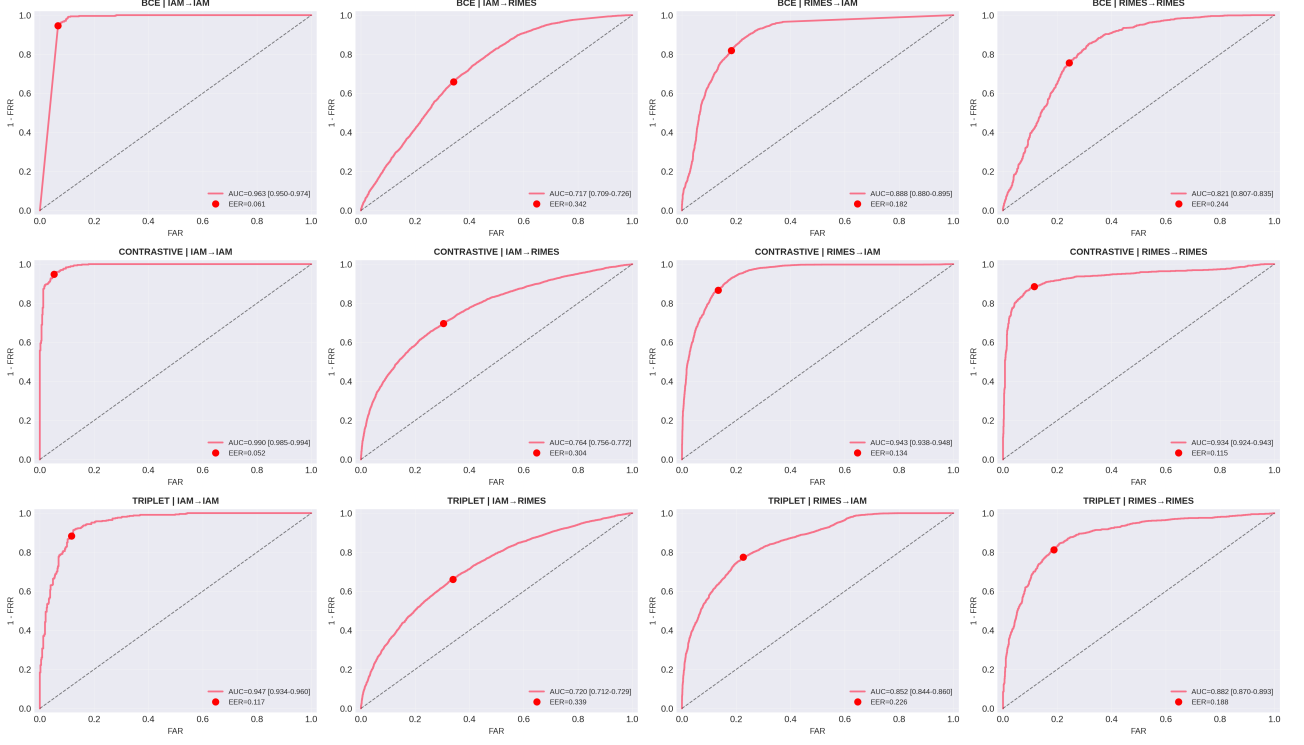


Fig. 7 ROC curves, ResNet-18, all losses/splits. Bootstrap 95% confidence intervals for AUC are reported in each panel legend.

Curve DET — backbone: resnet18

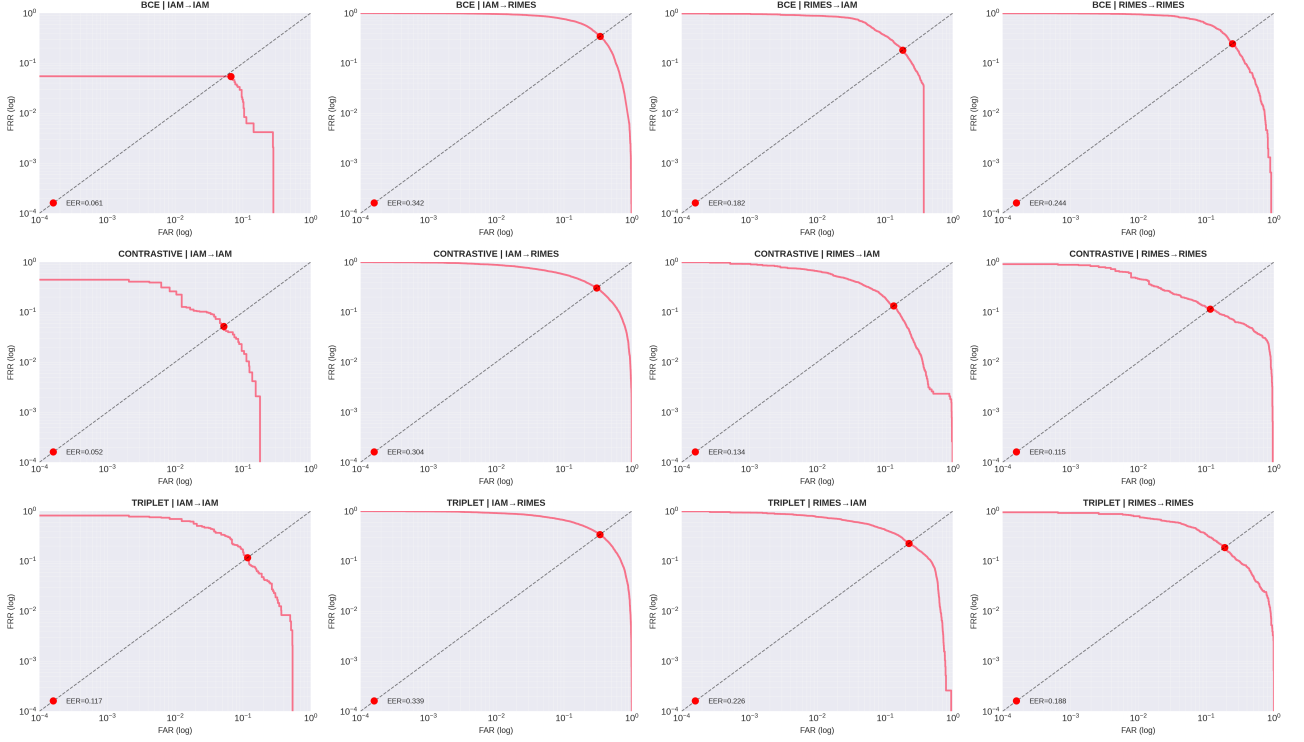


Fig. 8 DET curves (log-log), ResNet-18, all losses/splits.

producing the most favorable operating characteristics and the lowest EER (11.5% vs. 24.4% for BCE and 18.8% for Triplet). Overall, the DET plots corroborate the strong dependence of verification performance on the dataset pairing: the cross-dataset shift is particularly detrimental for IAM→RIMES, where all three losses exhibit substantially higher FRR across the operating range than in either same-dataset setting. Among the two cross-dataset splits, Contrastive provides the more favorable behavior on RIMES→IAM, although its performance still degrades markedly relative to its own same-dataset results under the IAM→RIMES shift.

5.9 Score distributions and the Gaussianity assumption

Shapiro–Wilk tests on all 72 genuine and 72 impostor score distributions reject normality universally at $\alpha = 0.05$ (largest observed p -value $\sim 2 \times 10^{-6}$, ResNet-18/Contrastive/RIMES→RIMES genuine scores); no distribution in the sweep is Gaussian-compatible. Consequently, d' /decidability are reported throughout this paper strictly as descriptive, rank-consistent indices rather than as parametric inferential statistics.

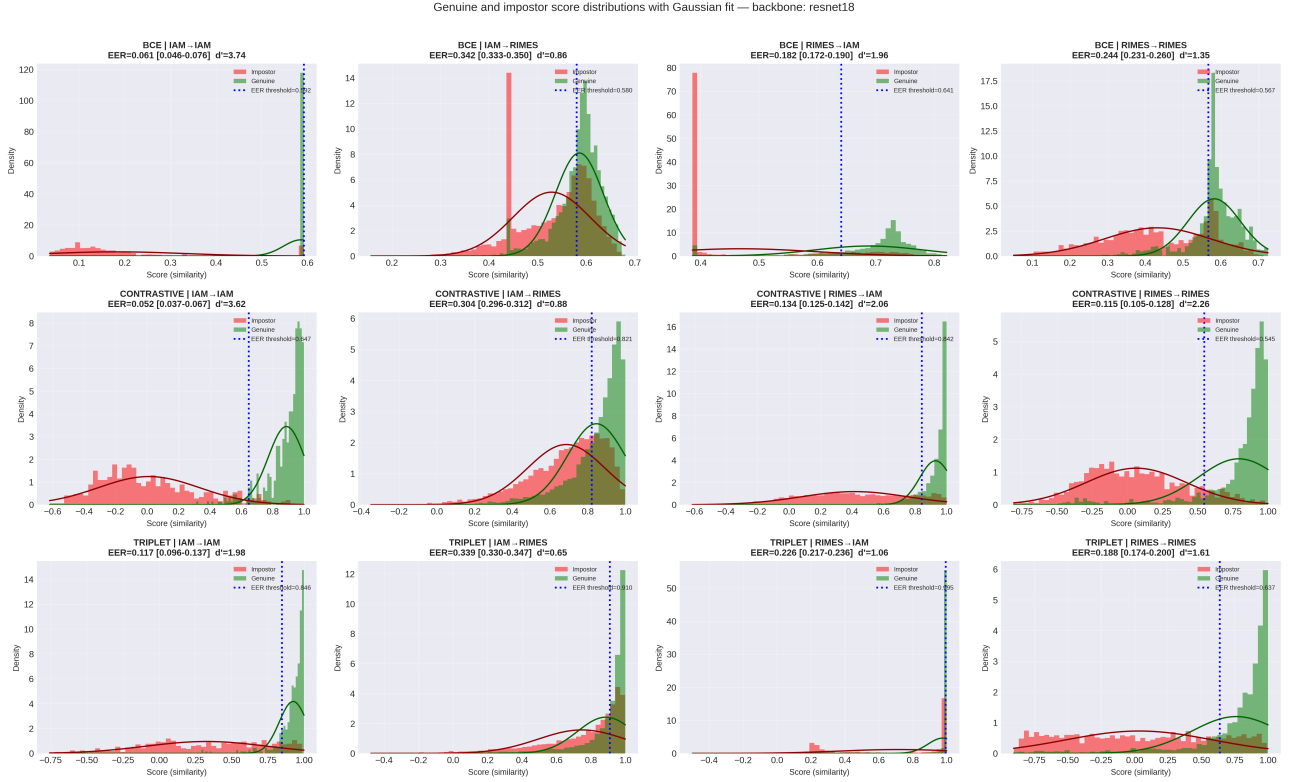


Fig. 9 Genuine (green)/impostor (red) scores with fitted Gaussian overlays, ResNet-18, all loss×split combinations. Dotted lines: EER threshold. None of the 24 distributions shown pass Shapiro–Wilk at $\alpha = 0.05$.

Visual inspection of Figure 9 shows that BCE/RIMES→IAM and BCE/RIMES→RIMES exhibit substantially more genuine-impostor overlap than the same-dataset IAM→IAM panel, consistent with their comparatively higher EER (18.2% and 24.4%, respectively, vs. 6.1% for BCE/IAM→IAM); this reflects a genuine loss of separability under BCE on these splits, though for ResNet-18 both configurations remain well above chance-level discrimination ($AUC = 0.888$ and 0.821 ; ResNet-18 does not exhibit any of the fully collapsed, near-chance BCE configurations documented for other backbones in Section 5.5). Contrastive and Triplet distributions, by contrast, display visible boundary-clustered mass near the similarity limits ± 1 , particularly for RIMES→IAM.

A further pattern visible in Figure 9 is that under BCE, several impostor distributions exhibit a narrow, isolated spike at a fixed low score (e.g. BCE/RIMES→IAM, impostor mass concentrated near score ≈ 0.39 ; BCE/IAM→RIMES, near ≈ 0.44), rather than a smooth unimodal shape – a possible signature of a degenerate embedding direction rather than genuine score variability. Under Contrastive and Triplet, by contrast, impostor distributions are broad and (generally) left-skewed while genuine distributions are (generally) right-skewed and compressed near the similarity ceiling (≈ 1.0), producing the asymmetric overlap responsible for the boundary-saturation effect noted above; this asymmetry is most pronounced for cross-dataset splits, and least pronounced for same-dataset splits, mirroring the EER ordering across splits (Tables 3–4).

5.10 Failure-case analysis

We inspect the two most relevant failure cases for the RIMES→IAM (ResNet-18 + Contrastive) configuration, a pair-level export of the top false accepts and false rejects.

- **Distribution-level patterns.** Two failure modes are visible for ResNet-18. First, BCE/IAM→RIMES shows genuine and impostor modes heavily overlapping around the same 0.5–0.6 score region (EER = 34.2%, $d' = 0.86$), the weakest separation among all four BCE panels and the closest ResNet-18 comes to the near-chance behavior of the fully collapsed models in Section 5.5 (none of which involve ResNet-18); BCE/RIMES→RIMES shows a milder version, with a broader impostor distribution encroaching on the genuine mode’s lower tail (EER = 24.4%, $d' = 1.35$) while the genuine peak stays distinct. Second, under Contrastive on both cross-dataset splits, a non-trivial fraction of impostor pairs saturate near the maximum cosine similarity (≈ 1.0), overlapping the genuine mode; this produces the long, shallow DET tail in Figure 8 and explains why GAR at strict FAR (Section 5.4) collapses to single digits on IAM→RIMES despite a moderate mean EER (32.2%).
- **Pair-level inspection (RIMES→IAM, ResNet-18 + Contrastive).** Figure 10 shows the top false accepts (scores 0.9986/0.9964, $\Delta \approx +0.155/+0.154$ above the 0.8424 threshold): different writers sharing slant, spacing, and regularity – confident errors driven by surface stylistic similarity the embedding does not disentangle from identity, consistent with the boundary-saturation pattern above. Figure 11 shows the top false rejects (scores $-0.3076/-0.2111$, $\Delta \approx -1.150/-1.054$ below threshold): the *same* writer across samples differing markedly in letter size, stroke pressure, and legibility.

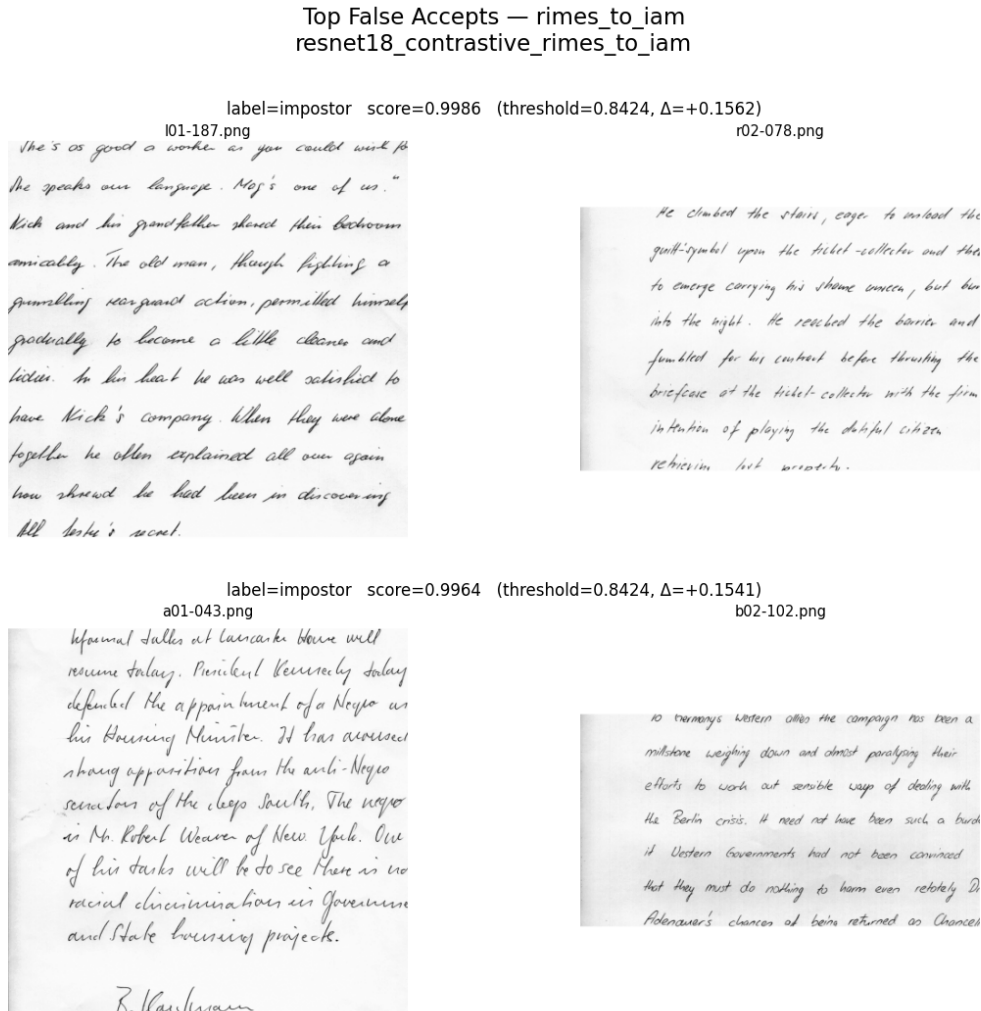


Fig. 10 Top false accepts, RIMES→IAM, ResNet-18 + Contrastive. Each row shows one impostor pair scored well above the EER threshold (0.8424), with the assigned score and the margin Δ above threshold.

This intra-writer inconsistency is not an IAM artifact specific to these two pairs: writer-to-sample counts in IAM are highly uneven (as few as one page for roughly half of the 657 writers, up to 59 for a single writer, see Subsection 4.3), so writer identity does not guarantee within-writer homogeneity of content, layout, or

Top False Rejects — rimes_to_iam resnet18_contrastive_rimes_to_iam

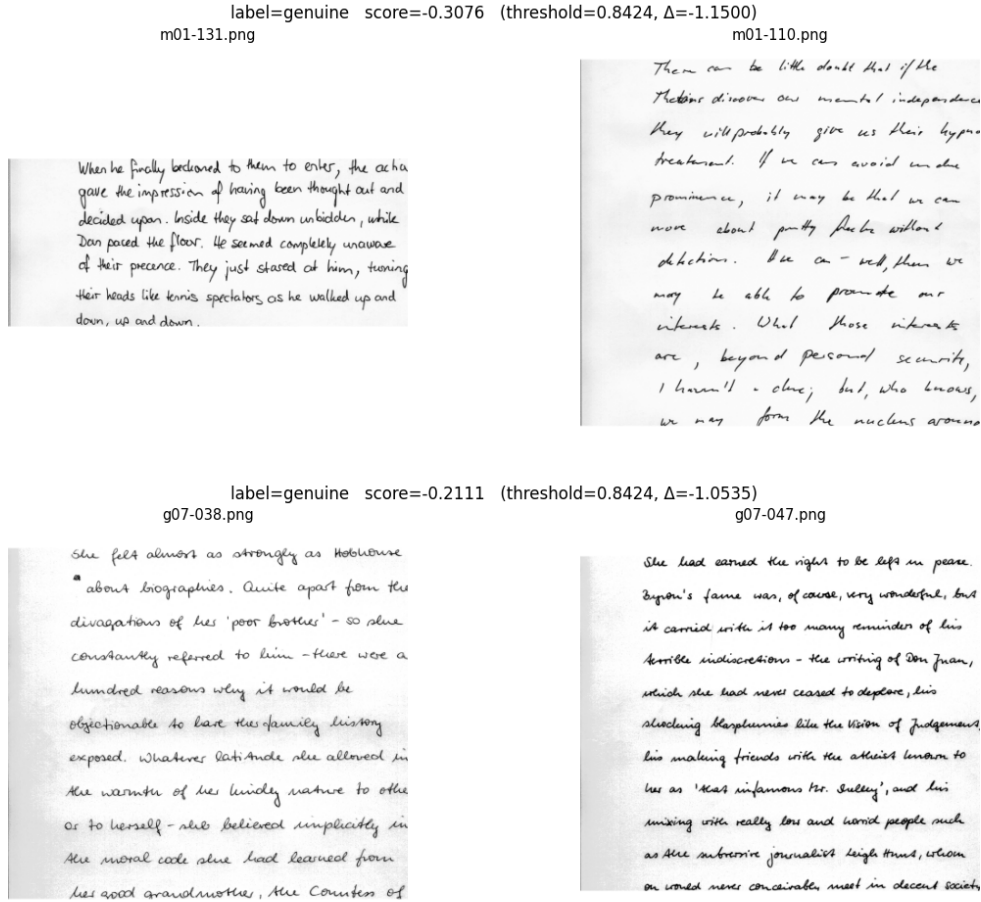


Fig. 11 Top false rejects, RIMES→IAM, ResNet-18 + Contrastive. Each row shows one genuine pair scored well below the EER threshold (0.8424), with the assigned score and the margin Δ below threshold.

scale. More fundamentally, forensic document-examination literature documents *multi-individuality of hand-writing*[26]: some writers maintain two or more distinct styles used spontaneously or deliberately, which can appear as different as two different people’s handwriting to an automated system trained on surface stylistic cues. This offers a mechanistic explanation for the Contrastive-over-BCE cross-dataset gap (Sections 5.3, 5.6): a representation invariant to stroke width, slant and spacing – properties BCE’s un-margined objective has less incentive to suppress – would better separate genuine pairs whose surface style varies while remaining sensitive to the subtler, writer-specific features (letter formation, proportions) that persist across such variation. Under this reading, false rejects and false accepts have distinct sources: the former is largely intra-writer style variability inherent to IAM’s uncontrolled-form collection protocol, the latter is embedding under-disentanglement of surface style from identity, and only the second is directly actionable via representation learning.

High intra-writer variability can contribute to both types of verification errors. Genuine pairs may be falsely rejected when stylistic differences make samples from the same writer appear dissimilar. Conversely, impostor pairs may be falsely accepted when superficial similarities make different writers appear similar. This latter error can arise indirectly from the model learning to associate substantially different writing styles with the same writer during training: by observing diverse samples from the same writer, the model is encouraged to map stylistically different instances to similar representations, but this may also reduce its ability to distinguish writers who share similar surface styles.

This inspection covers a single (backbone, loss, split) triple and the top-2 errors per failure mode; a systematic pair-level analysis across the full test set, and a check of whether errors concentrate on low-sample-count writers, is left to future work (Section 6.2).

5.11 Comparison with prior work

Direct benchmarking against the literature is constrained by three methodological gaps, so we position our results as a contextual reference rather than a leaderboard ranking (Table 11).

Task mismatch. Most CNN work on IAM and comparable corpora addresses closed-set identification [15, 16], reporting top-1 accuracy or mAP. These numbers are not commensurable with the open-set, pairwise EER/AUC used here: identification assumes every probe has an enrolled match and only ranks candidates, whereas verification must also reject unseen writers at a fixed threshold. The closest methodological neighbour, the triplet CNN of Keglevic et al. [20], is likewise a retrieval system evaluated by mAP.

Protocol mismatch. Comparable Siamese *verification* studies are typically evaluated on different scripts—Arabic, Chinese or Japanese—or under same-dataset splits only, so neither the data distribution nor the cross-dataset stress test we apply is reproduced. We are not aware of prior work reporting BCE, Contrastive and Triplet objectives under one fixed training recipe across a shared backbone set on IAM and RIMES, which is the specific comparison our study isolates.

Operating-point reporting. Prior studies rarely report GAR at strict FAR; a low EER can coexist with a sharp collapse in genuine acceptance at FAR = 0.1%, as our results show (Section 5.4). This limits comparability with the security-oriented regime and is a dimension we make explicit.

Table 11 Prior work on CNN-based writer identification/verification.

Method	Task	Dataset	Metric	Value
Fiel & Sablatnig [15]	Identification (closed-set)	IAM (train), ICDAR13/CVL (eval)	Acc./mAP	not comparable (task)
Christlein et al. [17]	Identification (closed-set)	ICDAR13, CVL	mAP	not comparable (task)
Keglevic et al. (triplet CNN) [20]	Retrieval / identification, triplet metric learning	ICDAR13	mAP	mAP 86.1% – not comparable (task)
Afzali et al. (CSCNN) [27]	Verification, Siamese + combined loss	IAM, IFN/ENIT	Accuracy, FAR, FRR	Partially comparable (different metrics/protocol)
Scribe verification [28]	Verification, same protocol family	Bamboo Slips, MCCD	AUC/EER-like	AUC 0.86–0.96, EER-like 7.8–21% – partially comparable (script differs)
Ours (best, Contrastive)	Verification	RIMES→IAM	AUC/EER	0.967 / 8.2% (EfficientNet-B0)
Ours (best, Contrastive)	Verification	IAM→IAM	AUC/EER	0.995 / 3.1% (MobileNetV3-Large)

Within these caveats, our best cross-dataset configuration (RIMES→IAM, EER = 8.2%) falls inside the range reported by the closest comparable Siamese-verification work (7.8–21%), and our same-dataset result (IAM→IAM, EER = 3.1%) is numerically lower than that range, although a direct comparison is not warranted, since the datasets and protocols differ. Rather than claiming state-of-the-art, our contribution is a controlled, uniformly-evaluated comparison: across every dataset split the metric-learning objectives dominate BCE, and Contrastive is the most consistent—a conclusion the heterogeneous prior literature, mixing tasks, scripts and protocols, does not allow one to draw directly.

6 Conclusion

This work presented HandVerify, a framework for offline handwriting verification based on deep Siamese networks. We systematically evaluated 72 (backbone, loss, dataset-pairing) configurations – six CNN backbones (ResNet-18/34, EfficientNet-B0/B1, MobileNetV3-Small/Large) trained under three loss paradigms (BCE, Contrastive, Triplet) and assessed on four train→test dataset pairings spanning IAM and RIMES – establishing baselines and providing practical guidance for verification system design. Each configuration was trained once, with a single random seed and a single writer-disjoint split; no repeated-fold cross-validation was performed.

The principal contributions are as follows: (i) a systematic comparison of three learning paradigms across six CNN backbones, under both same-dataset and cross-dataset evaluation; (ii) the introduction of two training strategies, epoch-wise dynamic negative resampling and selective backbone-layer freezing; (iii) a biometric evaluation using EER, AUC, decidability, and fixed-FAR operating points, with a best same-dataset EER of 3.1% (IAM→IAM, MobileNetV3-Large + Contrastive) and a best cross-dataset EER of 8.2% (RIMES→IAM, EfficientNet-B0 + Contrastive); and (iv) a paired statistical analysis (Wilcoxon signed-rank) confirming the consistency of the Contrastive-loss advantage across backbones and splits.

Overall, Contrastive Loss consistently outperformed BCE and Triplet Loss across nearly all backbones, splits, and evaluation regimes, at both the aggregate and best-single-model level, though its advantage was not uniform: gains were largest for backbones where BCE training was unstable (e.g., MobileNetV3-Small) and smallest for ResNet-18, the only backbone with no collapsed BCE configurations. Cross-dataset generalization remained substantially harder than same-dataset evaluation for all losses, particularly under strict low-FAR operating conditions.

6.1 Future Directions

Several avenues merit further investigation:

Statistical robustness. Multi-seed training and bootstrap confidence intervals would allow the point estimates reported here to be complemented with proper uncertainty quantification, and would strengthen the independence assumptions underlying the significance analysis.

Broader architectural coverage. Extending the sweep to transformer-based and attention-augmented architectures, and to additional backbone families, could clarify whether the observed loss-function ordering generalizes beyond the six CNN backbones considered here.

Advanced negative sampling. Online hard-negative mining and embedding-distance-based adaptive sampling may further improve Triplet Loss training beyond simple epoch-wise resampling.

Fine-grained failure analysis. A systematic, writer- and pair-level failure analysis across the full test sets – rather than the single-backbone, top-2-error inspection performed here – could clarify whether errors concentrate on low-sample-count writers or on genuinely multi-style writers.

Explainability. Visualization techniques such as Grad-CAM could elucidate the regions driving verification decisions, improving interpretability and user trust.

Embedding dimensionality analysis. A systematic study of the embedding dimensionality – e.g., via PCA-based intrinsic dimensionality estimation or a sweep over the output layer size – could reveal whether the current embedding size is over- or under-parameterized, and whether a more compact representation would preserve verification performance while reducing storage and matching cost.

Multi-modal integration. Combining offline handwriting with complementary biometric modalities (e.g., signatures, online writing dynamics) may improve overall system robustness.

Doddington zoo analysis. A per-writer categorization into biometric menagerie classes (goats, lambs, wolves, and extended classes such as doves, chameleons, phantoms and worms [29]) could reveal whether verification errors are broadly distributed across the writer population or concentrated in a small subset of problematic identities. Such an analysis requires aggregating genuine and impostor scores at the individual-writer level rather than pooling them across the test set, which was not retained in the current evaluation pipeline; combined with the failure-case inspection above, it could clarify whether the false accepts and false rejects observed in Section 5.10 are symptomatic of a few chronically difficult writers (consistent with the multi-individuality argument in Section 5.10) or are spread evenly across the population.

6.2 Limitations and Confounding Factors

- **Single seed.** Each configuration was trained with a single random seed; no per-model bootstrap confidence intervals were computed for this revision, so the reported figures are point estimates, not interval estimates, of evaluation uncertainty.
- **EER-threshold calibration.** Classification metrics (accuracy, precision, recall, F1; Table 10) were computed and evaluated on the same set, without an independent calibration split, and are therefore mildly optimistic.
- **Single-split same-dataset evaluation.** Same-dataset experiments employed a single train/test split rather than k -fold cross-validation, owing to computational and GPU-hour constraints on the Kaggle platform. The reported same-dataset performance may therefore be sensitive to the particular writer partition, and its variability across alternative splits could not be assessed within the present study.
- **Limited data augmentation.** The training pipeline employed only RandomResizedCrop, with a relatively narrow scale range of (0.9, 1.1), and RandomRotation with a maximum angle of 15° , adopted primarily due to computational constraints. Additional geometric and photometric perturbations (e.g., small translations, shear, brightness/contrast variation, mild Gaussian blur) were not evaluated and remain an open direction.
- **Cross-loss metric comparability.** Raw similarity scores are not directly comparable across loss functions in absolute terms (BCE yields sigmoid probabilities in $[0, 1]$; Contrastive/Triplet yield cosine similarities in $[-1, 1]$); accordingly, only rank-based metrics (AUC, EER, FAR/FRR) and the affine-invariant decidability index d' support the comparative claims made in this work.
- **Non-Gaussian score distributions.** Non-normality was confirmed for all 72 configurations (Section 5.9); d' /decidability are therefore used strictly as descriptive indices, not as parametric inferential statistics.
- **Non-independence in Wilcoxon testing.** The 24 paired observations analyzed in Section 5.6 share backbones, data splits, and single-seed training runs; the corresponding significance results should be interpreted as evidence of directional consistency rather than as 24 statistically independent replicates.
- **Granularity of failure-case analysis.** The qualitative analysis in Section 5.10 is restricted to distribution- and curve-level patterns for a single backbone (ResNet-18), plus a top-2-error pair-level inspection for a single (backbone, loss, split) triple; individual false-accept/ false-reject pairs, writer identifiers, and per-pair scores were not systematically retained. A finer-grained, image- and writer-level failure analysis is left to future work.

- **Scope of exhaustive validation.** The exhaustive impostor-pool validation reported in Section 5.7 covers a single (backbone, loss, split) triple (ResNet-18 + Contrastive, RIMES→IAM) and does not extend to the remaining three splits or to other backbone/loss combinations.
- **Training dynamics.** Per-configuration epoch counts and stopping criteria (early stopping vs. epoch cap) were not available in consolidated form for this revision; raw logs were inspected directly only for a subset of BCE/EfficientNet-B1 runs.

Acknowledgements. The authors would like to thank Kaggle for providing access to GPU computational resources, which enabled the training and evaluation of the models presented in this work.

Declarations

Funding. Not applicable.

Conflict of interest. The authors declare no competing interests.

Ethics approval. Not applicable.

Data availability. The datasets used in this study are publicly available at the dataset links: [RIMES](#) and [IAM](#)

Code availability. The HandVerify framework is implemented in PyTorch and it is publicly available on [GitHub](#) under the MIT License.

Author contributions. Both authors contributed equally to this work. L.A. designed the framework architecture and conducted the training and testing experiments. E.G. implemented the application design, deployed the application, verified its correct functionality. Both authors wrote and reviewed the manuscript.

Appendix A Implementation Details

A.1 Preprocessing Pipeline Details

This appendix provides the full mathematical formulation of the preprocessing pipeline summarized in Section 4.

A.1.1 Binarization as an Auxiliary Mask

Binarization is used exclusively to *compute* crop coordinates and content masks. Concretely, each image is first converted to grayscale and, from it, an Otsu-thresholded binary version is derived and used only as a guide for locating text regions. Otsu’s method determines the optimal threshold t^* by maximizing inter-class variance:

$$t^* = \arg \max_t \sigma_B^2(t) = \arg \max_t \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2 \quad (\text{A1})$$

where $\omega_0(t)$ and $\omega_1(t)$ represent background and foreground class probabilities, and $\mu_0(t)$, $\mu_1(t)$ are their respective means. The resulting binary mask has text pixels set to 0 (black) and background to 255 (white), and is used downstream for: (i) detecting IAM guide lines, (ii) locating left-margin black columns and top/bottom black rows, and (iii) computing tight-crop bounding boxes. All crop coordinates derived from the binary mask are then applied to the corresponding grayscale image, which is what is actually cropped, resized, and written to the output dataset.

A.1.2 Tight Cropping

To eliminate excessive whitespace while preserving text content, we compute tight crop coordinates using axial projections on the binary mask, and apply the resulting bounding box to the grayscale image. For a binary image I of size $h \times w$, we invert it ($I_{\text{inv}} = 255 - I$) and compute row and column projections:

$$S_{\text{row}}(i) = \sum_{j=1}^w \mathbb{I}[I_{\text{inv}}(i, j) > 0], \quad S_{\text{col}}(j) = \sum_{i=1}^h \mathbb{I}[I_{\text{inv}}(i, j) > 0] \quad (\text{A2})$$

Adaptive thresholds determine content-bearing rows and columns, based on a minimum ink-density ratio ρ :

$$\tau_{\text{row}} = \max(10, \lfloor \rho \cdot w \rfloor), \quad \tau_{\text{col}} = \max(10, \lfloor \rho \cdot h \rfloor) \quad (\text{A3})$$

By default $\rho = 0.018$ (used for RIMES line processing); for IAM full-form images we use a slightly stricter $\rho = 0.02$ to compensate for residual guide-line artifacts left after region extraction.

The bounding box is defined by:

$$y_1 = \min\{i : S_{\text{row}}(i) > \tau_{\text{row}}\}, \quad y_2 = \max\{i : S_{\text{row}}(i) > \tau_{\text{row}}\} \quad (\text{A4})$$

$$x_1 = \min\{j : S_{\text{col}}(j) > \tau_{\text{col}}\}, \quad x_2 = \max\{j : S_{\text{col}}(j) > \tau_{\text{col}}\} \quad (\text{A5})$$

A 3-pixel internal trim, applied only along the row axis, removes residual guide lines: $y_1 \leftarrow \min(y_1 + 3, y_2 - 1)$ and $y_2 \leftarrow \max(y_2 - 3, y_1 + 1)$, while x_1, x_2 are left untouched. This is followed by 5-pixel padding on *all four* sides to prevent text from touching borders. The resulting coordinates (y_1, y_2, x_1, x_2) are then used to crop the grayscale image, not the binary mask.

A.1.3 Aspect-Preserving Resize

Images are normalized to 448×448 pixels by first centering them in a square canvas of side $s = \max(h, w)$ with offsets:

$$\text{offset}_y = \lfloor (s - h)/2 \rfloor, \quad \text{offset}_x = \lfloor (s - w)/2 \rfloor \quad (\text{A6})$$

The canvas is then resized to 448×448 using area interpolation, preserving the original aspect ratio without distortion.

A.1.4 IAM Dataset Preprocessing

IAM forms contain pre-printed guide lines that must be removed. We detect horizontal lines using morphological opening (erosion followed by dilation, 2 iterations) with a kernel of width $k_w = \max(30, \lfloor w/15 \rfloor)$:

$$\text{Opening}(I, K) = \text{Dilate}(\text{Erode}(I, K), K) \quad (\text{A7})$$

Valid lines satisfy: $w_c > 0.3w$, $h_c < 20$, and area > 100 pixels. Detected lines are sorted top-to-bottom and the handwritten region is selected according to how many valid lines are found:

- **Three or more lines detected:** the region is extracted between the second and third detected lines,

$$y_{\text{start}} = y_{\text{bottom}}^{(2)} + 8, \quad y_{\text{end}} = y_{\text{top}}^{(3)} - 8 \quad (\text{A8})$$

- **Exactly two lines detected:** the region is extracted between them,

$$y_{\text{start}} = y_{\text{bottom}}^{(1)} + 8, \quad y_{\text{end}} = y_{\text{top}}^{(2)} - 8 \quad (\text{A9})$$

- **Fewer than two lines detected:** a secondary detection pass estimates guide-line positions from a smoothed row-density profile. The row-wise ink density is smoothed with a uniform filter (window size 5), and rows exceeding the 95th percentile of the smoothed profile are grouped into contiguous line segments (gap tolerance of 5 pixels). If this secondary pass yields three or more segments, or exactly two, the same rules above are re-applied to the segments; otherwise the pipeline falls back to a fixed region.
- **Fallback:** if no reliable lines are found by either pass,

$$y_{\text{start}} = \lfloor 0.22h \rfloor, \quad y_{\text{end}} = \lfloor 0.85h \rfloor \quad (\text{A10})$$

As a final safeguard, if the extracted region is empty or smaller than 30×50 pixels (height $\times$ width), extraction reverts to the fixed fallback region above. Guide-line detection and region selection are performed on the binary mask, but the corresponding rows are extracted from *both* the grayscale and binary versions of the image, keeping them spatially aligned for the subsequent steps. After region extraction, black column removal (removing columns with $> 50\%$ black pixels from the left margin, computed on the binary region) and tight cropping (coordinates computed on the binary region with $\rho = 0.02$, Appendix A.1.2) are applied to the grayscale region before final resizing. The binary region itself is discarded after this point and never saved. Figure A1 illustrates this pipeline end to end, from the original grayscale form to the final cropped and resized image.

Two additional sanity checks guard against degenerate cases. First, immediately after y_{start} and y_{end} are determined by any of the branches above, if $y_{\text{start}} \geq y_{\text{end}}$ (an invalid, empty, or inverted range) the pipeline discards that estimate and reverts directly to the fixed fallback $\lfloor 0.22h, 0.85h \rfloor$. Second, whichever values of $y_{\text{start}}, y_{\text{end}}$ are ultimately used are clipped to the valid image range, $y_{\text{start}} \leftarrow \max(0, y_{\text{start}})$ and $y_{\text{end}} \leftarrow \min(h, y_{\text{end}})$, before the region is sliced from the image. This clipping is independent of, and precedes, the minimum-size safeguard described above.

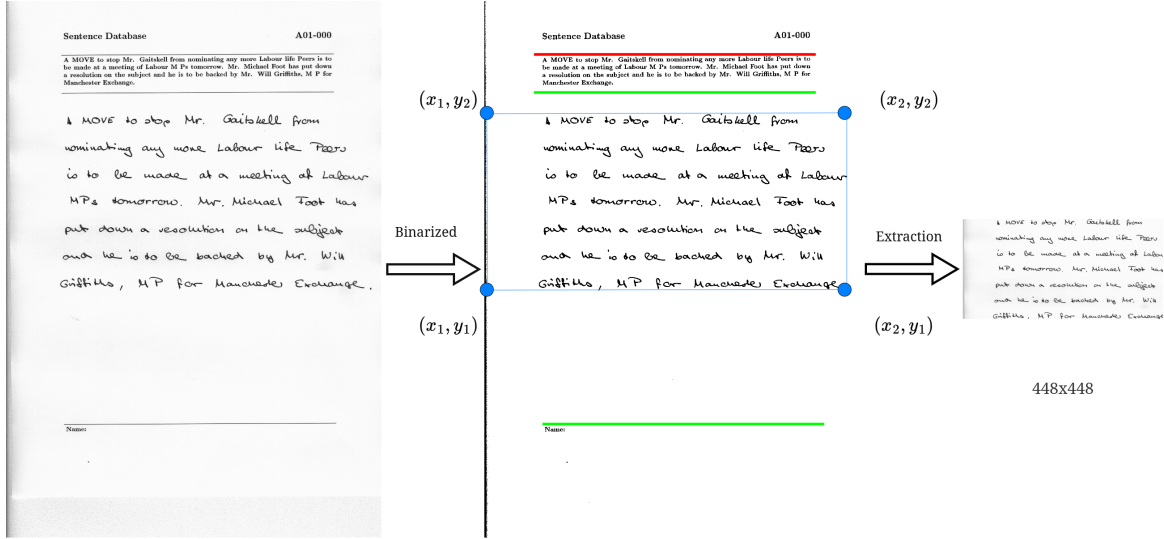


Fig. A1 Preprocessing of an IAM form: the grayscale image (left) is binarized (center) to detect guide lines (red/green, detected by erosion/dilation) and the handwritten text region (blue box); this region is then extracted from the original grayscale image, cropped, and resized to 448×448 pixels (right).

A.1.5 RIMES Dataset Preprocessing

RIMES consists of individual text lines that are processed and stacked vertically to form 448×448 batches. Each line is first converted to grayscale, and an auxiliary Otsu binary mask is derived from it (as described above) purely to guide cropping; only the grayscale line is cropped, resized, and eventually stacked into the output batch. Each line undergoes:

1. **Vertical trim:** Removal of rows with $> 50\%$ black pixels from top and bottom, computed on the binary mask and applied identically to both the mask and the grayscale line
2. **Tight cropping:** Bounding-box coordinates computed on the binary mask via the projection-based algorithm ($\rho = 0.018$, Appendix A.1.2), applied to crop the grayscale line
3. **Height normalization:** For each writer, the minimum observed line height across all their (cropped) lines is computed and clamped to the range $[20, 32]$ pixels, $h_{\min} = \text{clip}(\min_l h_l, 20, 32)$. All lines from that writer are then resized to height $h_{\min}$, preserving aspect ratio:

$$w' = \lfloor w \cdot h_{\min} / h \rfloor \quad (\text{A11})$$

If the resulting width w' exceeds the target canvas width (448 px), the line is instead scaled down so that $w' = 448$ and $h' = \lfloor h \cdot 448 / w \rfloor \leq h_{\min}$. Each resized line is placed on a fixed-size $h_{\min} \times 448$ white canvas, left-aligned and vertically centered with offset $\lfloor (h_{\min} - h') / 2 \rfloor$.

4. **Batching:** Height-normalized lines are vertically stacked into groups of $N_{\text{batch}} = \lfloor 448 / 32 \rfloor - 9 = 5$ lines per batch, producing 448×448 canvases per writer. The last batch for a writer is padded with white space if fewer than N_{batch} lines remain. Figure A2 illustrates this pipeline, from original lines to tight-cropped lines and their grouping into stacked 448×448 batches.

Lines exceeding 896 pixels in width (twice the target size) are split at the midpoint before height normalization, to prevent excessive aspect ratio distortion. This produces uniform-height stacked representations suitable for Siamese network training. Note that with $N_{\text{batch}} = 5$ lines of at most 32 px height each, a batch occupies at most 160 of the 448 available vertical pixels; the remaining space is white padding.

A.2 Dataset Statistics: Distribution Figures

Figure A3 shows the per-author image count distributions for both datasets after preprocessing, complementing the summary statistics of Table 1. The IAM histogram (top) is dominated by a large spike at 1–2 images per author, with a long, sparse tail extending up to the outlier author with 59 images. The RIMES histogram is comparatively broader and shifted toward higher counts, peaking around 3 batches per author and extending up to 10, reflecting a more even but still right-skewed spread. The boxplot comparison (bottom) further highlights this asymmetry: IAM’s interquartile range is narrow and close to zero, with the extreme outlier and long upper whisker revealing a much heavier tail, while RIMES shows a wider, more centrally distributed spread with a shorter relative tail.

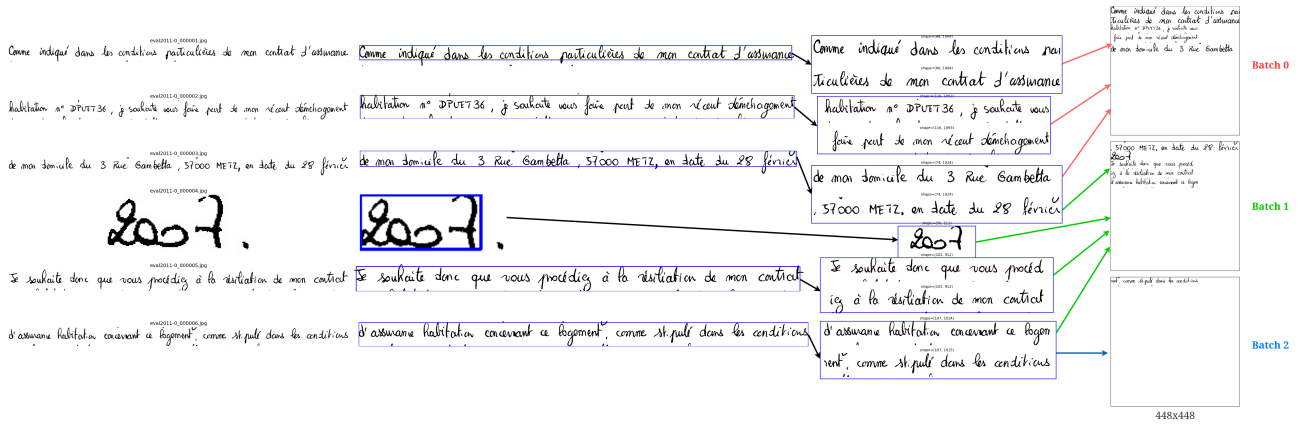


Fig. A2 Preprocessing of RIMES text lines (for a given author). Each original line (left, e.g. `eval2011-0.000001.jpg`) is binarized via Otsu thresholding and tight-cropped (blue bounding boxes, center) using the projection-based algorithm; the corresponding grayscale crop is then extracted (right of center). Cropped lines belonging to the same writer are grouped (colored arrows) and vertically stacked into fixed-size 448×448 canvases (right), with unused space padded in white.

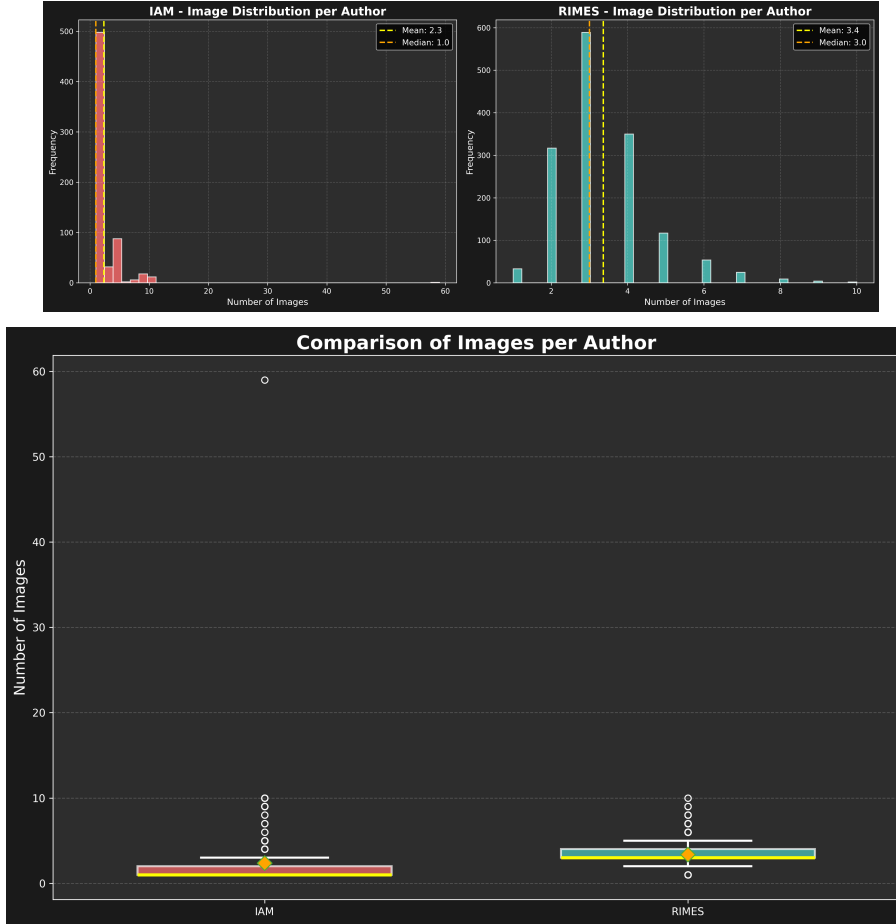


Fig. A3 Distribution of images per author for the preprocessed IAM and RIMES datasets. **Top:** per-dataset histograms with mean (yellow) and median (orange) overlaid, showing the strong right skew of IAM compared to the broader, more centrally distributed RIMES spread. **Bottom:** side-by-side boxplot comparison, highlighting the extreme outlier author in IAM (59 images) against the wider but shorter-tailed RIMES distribution (max 10 images).

A.3 Hyperparameter Configuration

Table A1 summarizes the hyperparameter configuration used across the 72-model sweep, including values already discussed in the main text (Section 4) and additional training-setup details not otherwise reported, such as maximum epochs, embedding dimension, and data-loading workers. Loss-specific margins follow Section 3: $m = 1.0$ for Contrastive and $m = 0.5$ for Triplet.

Table A1 Hyperparameter configuration shared across the 72 model configurations. Loss-specific settings are indicated where applicable.

Hyperparameter	Configuration
Batch size	16
Maximum epochs	50
Early stopping	Patience = 5
Image size	448 × 448 pixels
Embedding dimension	32
Margin	1.0 (Contrastive), 0.5 (Triplet)
Positive ratio	0.5
Dropout	0.2
Frozen encoder layers	3
Negative resampling	Every epoch (all losses)
Data-loading workers	4

A.4 Training Dynamics

Figure A4 reports training and validation loss curves for the ResNet-18 backbone across all 12 loss×split combinations. Across nearly all configurations, training loss decreases monotonically, consistent with stable optimization; however, validation-loss behavior differs substantially by loss function and split. Under BCE, validation loss tracks training loss relatively closely on the same-dataset splits (IAM→IAM, RIMES→RIMES), while the cross-dataset splits show an early divergence between the two curves (e.g., IAM→RIMES), consistent with the same-vs-cross-dataset generalization gap discussed in Section 5.3. Under Contrastive and, more markedly, under Triplet Loss, validation loss is considerably noisier and frequently non-monotonic, with pronounced oscillations and, in several cases (e.g., Triplet on IAM→IAM and IAM→RIMES), no clear downward trend despite steadily decreasing training loss – indicative of a widening train-validation gap rather than smooth joint convergence. Training run lengths also vary considerably across configurations, from as few as 6 epochs (Triplet, RIMES→RIMES) to over 40 (BCE, IAM→IAM), reflecting the early-stopping criterion (patience 5, Appendix A.3) being triggered at different points depending on the stability of the validation signal for each configuration.

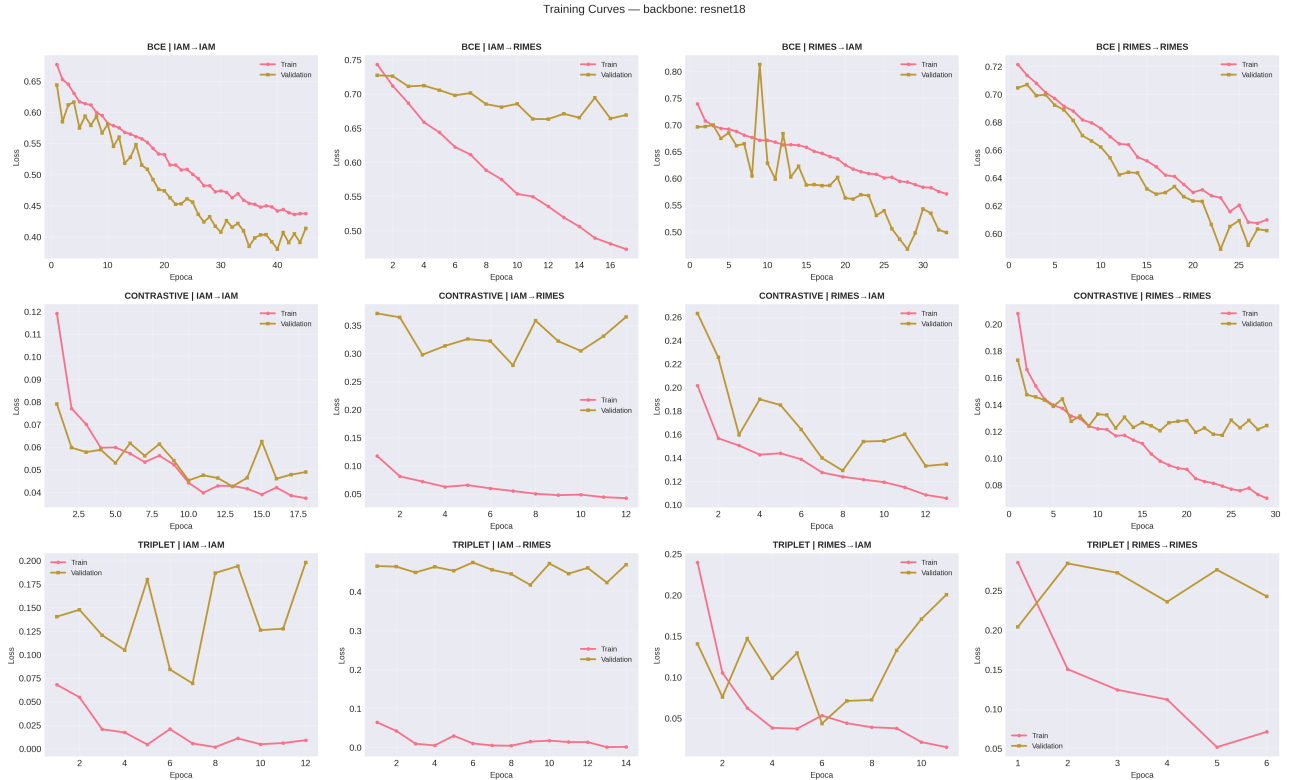


Fig. A4 Training and validation loss curves, ResNet-18, all 12 loss×split combinations.

References

- [1] Koch, G., Zemel, R., Salakhutdinov, R.: Siamese neural networks for one-shot image recognition. In: ICML Deep Learning Workshop, vol. 2. Lille, France (2015). <https://www.cs.toronto.edu/~zemel/documents/oneshot1.pdf>
- [2] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. IEEE, Las Vegas, NV, USA (2016). <https://doi.org/10.1109/CVPR.2016.90>
- [3] Tan, M., Le, Q.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: Proceedings of the 36th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 97, pp. 6105–6114. PMLR, ??? (2019). <https://doi.org/10.48550/arXiv.1905.11946>
- [4] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V., Adam, H.: Searching for MobileNetV3. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1314–1324 (2019). <https://doi.org/10.48550/arXiv.1905.02244>
- [5] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255. IEEE, ??? (2009). <https://doi.org/10.1109/CVPR.2009.5206848>
- [6] Hadsell, R., Chopra, S., LeCun, Y.: Dimensionality reduction by learning an invariant mapping. In: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), vol. 2, pp. 1735–1742. IEEE Computer Society, ??? (2006). <https://doi.org/10.1109/CVPR.2006.100>
- [7] Schroff, F., Kalenichenko, D., Philbin, J.: FaceNet: A unified embedding for face recognition and clustering. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 815–823 (2015). <https://doi.org/10.1109/CVPR.2015.7298682>
- [8] <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>. [Accessed 07-09-2026]
- [9] RIMES-Lines: Handwritten OCR Line-Level Dataset — kaggle.com. <https://www.kaggle.com/datasets/chaimaourgani/handwritten2text-training-dataset>. [Accessed 07-09-2026]
- [10] Bensefia, A., Paquet, T., Heutte, L.: A writer identification and verification system. Pattern Recognition Letters **26**(13), 2080–2092 (2005)
- [11] Said, H.E.S., Tan, T.N., Baker, K.D.: Personal identification based on handwriting. Pattern Recognition **33**(1), 149–160 (2000)
- [12] Schomaker, L., Bulacu, M.: Automatic writer identification using connected-component contours and edge-based features of uppercase western script. IEEE Transactions on Pattern Analysis and Machine Intelligence **26**(6), 787–798 (2004)
- [13] Bulacu, M., Schomaker, L.: Text-independent writer identification and verification using textural and allo-graphic features. IEEE Transactions on Pattern Analysis and Machine Intelligence **29**(4), 701–717 (2007) <https://doi.org/10.1109/TPAMI.2007.1009>
- [14] Plamondon, R., Srihari, S.N.: Online and off-line handwriting recognition: A comprehensive survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **22**(1), 63–84 (2000) <https://doi.org/10.1109/34.824821>
- [15] Fiel, S., Sablatnig, R.: Writer identification and retrieval using a convolutional neural network. In: Computer Analysis of Images and Patterns (CAIP). LNCS, vol. 9257, pp. 26–37. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-23117-4_3
- [16] Christlein, V., Bernecker, D., Maier, A., Angelopoulou, E.: Offline writer identification using convolutional neural network activation features. In: Pattern Recognition (GCPR). LNCS, vol. 9358, pp. 540–552. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-24947-6_45
- [17] Christlein, V., Gropp, M., Fiel, S., Maier, A.: Unsupervised feature learning for writer identification and

- writer retrieval. In: 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), pp. 991–997 (2017)
- [18] He, S., Schomaker, L.: Fragnet: Writer identification using deep fragment networks. *IEEE Transactions on Information Forensics and Security* **15**, 3013–3022 (2020)
 - [19] Bromley, J., Guyon, I., LeCun, Y., Säckinger, E., Shah, R.: Signature verification using a "Siamese" time delay neural network. In: *Advances in Neural Information Processing Systems 6 (NIPS 1993)*, pp. 737–744. Morgan Kaufmann, ??? (1994). Presentato al NIPS 1993 (Denver, CO); atti pubblicati nel 1994. Vol. 6 della serie *Advances in NIPS*.
 - [20] Keglevic, M., Fiel, S., Sablatnig, R.: Learning features for writer retrieval and identification using triplet CNNs. In: *16th International Conference on Frontiers in Handwriting Recognition (ICFHR)*, pp. 211–216. IEEE, ??? (2018). <https://doi.org/10.1109/ICFHR-2018.2018.00045>
 - [21] Mansfield, A.J., Wayman, J.L.: Best practices in testing and reporting performance of biometric devices. Technical Report CMSC 14/02, National Physical Laboratory (2002)
 - [22] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. *Proceedings of Machine Learning Research*, vol. 37, pp. 448–456. PMLR, Lille, France (2015). <https://doi.org/10.48550/arXiv.1502.03167>
 - [23] He, H., Garcia, E.A.: Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* **21**(9), 1263–1284 (2009) <https://doi.org/10.1109/TKDE.2008.239>
 - [24] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: *International Conference on Learning Representations (ICLR)* (2019). <https://doi.org/10.48550/arXiv.1711.05101>
 - [25] Rosenblum, M., Garrett, B.L., Scurich, N., *et al.*: Incorrect statistical reasoning in Guyll et al. leads to biased claims about strength of forensic evidence. *Proceedings of the National Academy of Sciences* (2024) <https://doi.org/10.1073/pnas.2315431121>
 - [26] Moszczynski, J.: The multi-individuality of handwriting. *Forensic Science International* **294**, 4–10 (2019) <https://doi.org/10.1016/j.forsciint.2018.10.030>
 - [27] Afzali, P., Rezapour, A., Rezaee Jordehi, A.: Leveraging deep feature learning for handwriting biometric authentication. *International Journal of Research in Industrial Engineering* **13**(1), 88–103 (2024)
 - [28] Wang, H., Li, M., Liu, B., *et al.*: Tsinghua bamboo slip scribe verification using siamese networks. *npj Heritage Science* **14**, 147 (2026) <https://doi.org/10.1038/s40494-026-02416-8>
 - [29] Yager, N., Dunstone, T.: The biometric menagerie. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **32**(2), 220–230 (2010) <https://doi.org/10.1109/tpami.2008.291>